



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

ANALIZA PARADIGMELOR DE ANOMALY DETECTION PE DIFERITE SETURI DE DATE

Absolvent

Savin Radu-Andrei

Coordonator științific

Paul Irofti

București, iunie 2021

Rezumat

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce vitae eros sit amet sem ornare varius. Duis eget felis eget risus posuere luctus. Integer odio metus, eleifend at nunc vitae, rutrum fermentum leo. Quisque rutrum vitae risus nec porta. Nunc eu orci euismod, ornare risus at, accumsan augue. Ut tincidunt pharetra convallis. Maecenas ut pretium ex. Morbi tellus dui, viverra quis augue at, tincidunt hendrerit orci. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam quis sollicitudin nunc. Sed sollicitudin purus dapibus mi fringilla, nec tincidunt nunc eleifend. Nam ut molestie erat. Integer eros dolor, viverra quis massa at, auctor.

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce vitae eros sit amet sem ornare varius. Duis eget felis eget risus posuere luctus. Integer odio metus, eleifend at nunc vitae, rutrum fermentum leo. Quisque rutrum vitae risus nec porta. Nunc eu orci euismod, ornare risus at, accumsan augue. Ut tincidunt pharetra convallis. Maecenas ut pretium ex. Morbi tellus dui, viverra quis augue at, tincidunt hendrerit orci. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam quis sollicitudin nunc. Sed sollicitudin purus dapibus mi fringilla, nec tincidunt nunc eleifend. Nam ut molestie erat. Integer eros dolor, viverra quis massa at, auctor.

Cuprins

1	Introducere	6
1.1	Context și Motivație	6
1.2	Structura Lucrării	7
2	Preliminarii	7
2.1	Proiectarea și Abordarea Cercetării	7
2.1.1	Ipoteze și Comportament Așteptat	8
2.2	Algoritmi de Detecție a Anomaliilor	8
2.2.1	One-Class SVM	8
2.2.2	Isolation Forest	9
2.2.3	Autoencoder Feedforward (MLP)	9
2.2.4	Autoencoder LSTM	10
2.2.5	LSTM Predictiv	10
2.2.6	Autoencodere Bazate pe Transformer	10
2.3	Seturile de Date	11
2.3.1	Credit Card Fraud Detection (Setul de Date A)	11
2.3.2	Yahoo Webscope S5 (Setul de Date B)	12
2.3.3	CIC-IDS2017 (Setul de Date C)	12
3	Methods and Experimental Setup	13
3.1	Preprocesare	13
3.1.1	Preprocesare Tabelară (Credit Card)	13
3.1.2	Crearea de Ferestre pentru Serii de Timp (Yahoo S5)	14
3.1.3	Preprocesarea Fluxurilor de Rețea (CIC-IDS2017)	15
3.2	Protocolul de Evaluare	16
3.2.1	Metrici Raportate	16
3.2.2	Selecția Pragului	16
3.2.3	Fără Protocolul <i>Point Adjustment</i>	17
4	Experimental Results	17
4.1	Detectarea Fraudei cu Cardul de Credit (Setul de Date A)	17
4.1.1	Comparație Generală	17
4.1.2	Discuție	17
4.2	Yahoo Webscope S5 (Setul de Date B)	19
4.2.1	A1 – Anomalii Punctuale (Metrici Reale de Server)	19

4.2.2	A3 – Anomalii Contextuale/Sezoniere	20
4.2.3	A4 – Anomalii de Tip Schimbare-de-Nivel	20
4.2.4	Verificare de Referință Trivială (Doar Yahoo)	21
4.2.5	Discuție	22
4.3	CIC-IDS2017 (Setul de Date C)	23
4.3.1	Comparație Generală	23
4.3.2	Detaliiere pe Tip de Atac	25
4.3.3	Discuție	26
4.4	Exploratory Data Analysis	27
4.4.1	Credit Card Fraud Detection	27
4.4.2	Yahoo Webscope S5	28
4.4.3	CIC-IDS2017	29
5	Explainability Case Studies	30
5.1	Reconstruction-Error Localization on Yahoo A3	30
5.1.1	Initial Assumption	31
5.1.2	Qualitative Check: Per-Timestep Error Heatmaps	31
5.1.3	Quantitative Verification: Gini Coefficient of Error Concentration	33
5.2	Disjoint Feature Regimes: AE+Context vs. Temporal IForest	34
5.2.1	Temporal IForest: Initial Assumption	35
5.2.2	Temporal IForest: Score Validity Under Last-Flow Labeling	35
5.2.3	Temporal IForest: SHAP Attribution Across the Window	35
5.2.4	AE+Context: SHAP Dominated by Host-Context Features	37
5.2.5	Building the Combined Score	39
5.2.6	Two Fusion Rules, Two Normalizations	39
5.2.7	Per-Class Coverage vs. Aggregate AP	40
6	Concluzii	41
1	Alegeri legate de Hiperparametri și Antrenare	41
1.1	Isolation Forest	42
1.2	One-Class SVM	42
1.3	Autoencoder Feedforward (MLP)	43
1.4	LSTM Predictiv	43
1.5	Autoencoder LSTM	44
1.6	Bottleneck Transformer Autoencoder	44
1.7	Causal Transformer	44
2	Protocolul de Evaluare: Metrici și Rezultate Complete	45
2.1	Metrici Raportate	45

2.2	Rezultate Complete – Credit Card Fraud Detection	46
2.3	Rezultate Complete – Yahoo Webscope S5	46
2.4	Rezultate Complete – CIC-IDS2017	47
3	Compoziția pe Clase de Atac a Setului de Test Temporal CIC-IDS2017	47

Capitolul 1

Introducere

1.1 Context și Motivație

Această lucrare este un studiu experimental în învățarea automată, mai precis în subdomeniul *detectiei de anomalii* (*anomaly detection*) – sarcina de a detecta puncte, evenimente sau structuri care deviază de la normalitate. Domeniul este unul larg și acoperă o varietate de probleme, dintre care în această lucrare sunt abordate: detectia fraudelor în tranzacții bancare, detectia atacurilor asupra rețelelor de calculatoare și detectia defecțiunilor în metricele monitorizate ale serverelor.

O primă distincție care trebuie făcută este cea dintre *outlier detection* și *novelty detection*. Prima presupune antrenarea unui model pe date care conțin deja anomalii, cu speranța de a le identifica ulterior, în timp ce a doua presupune antrenarea exclusiv pe date considerate normale și evaluarea capacității modelului de a detecta, la testare, devieri de la noțiunea de normalitate astfel învățată. Această lucrare adoptă abordarea de tip *novelty detection*, întrucât acesta este cadrul în care sunt implementate în practică cazurile de utilizare menționate mai sus (un model de detectie a fraudelor, de exemplu, nu este antrenat pe cazuri confirmate de fraudă). În plus, această alegere ajută la formularea scopului lucrării printr-o singură întrebare: în ce măsură modul în care un model reprezintă structura latentă a datelor considerate „normale” determină tipurile de anomalii pe care acesta le poate, sau nu, detecta?

Motivația, cât și scopul acestei lucrări sunt, așadar, căutarea unui răspuns la această întrebare și abordarea ulterioară a ipotezelor formate în urma rezultatelor experimentale.

Contribuția proprie a acestei lucrări constă, așadar, în:

1. un pipeline standardizat de preprocesare, antrenare și evaluare, aplicat uniform pe trei seturi de date diferite și pe până la treisprezece configurații de modele și seturi de caracteristici (*features*);
2. o analiză sistematică a performanței diferitelor paradigme în funcție de tipul setului de date; și
3. o serie de studii de caz de explicabilitate, menite să testeze direct ipoteze formulate în secțiunea de discuții a rezultatelor, folosind hărți termice ale erorii de reconstrucție, o măsură de concentrație bazată pe coeficientul Gini și valori de atribuire SHAP, în locul unor concluzii bazate exclusiv pe metrice agregate.

1.2 Structura Lucrării

Restul lucrării este organizat în cinci capitole.

Capitolul 2 (*Preliminarii*) introduce fundalul teoretic necesar pentru restul lucrării: detalii despre funcționarea fiecărui model testat, paradigma căreia îi aparține și seturile de date folosite.

Capitolul 3 (*Methods and Experimental Setup*) descrie pipeline-urile de preprocesare, alegerile de hiperparametri și de arhitectură făcute pentru fiecare model, precum și protocolul de evaluare – inclusiv metricile raportate și deciziile metodologice.

Capitolul 4 (*Experimental Results*) prezintă rezultatul fiecărui experiment, organizat pe seturi de date, împreună cu o secțiune de discuții pentru fiecare set de date, care interpretează rezultatele prin raportare la așteptările stabilite în capitolul de Preliminarii.

Capitolul 5 (*Explainability Case Studies*) revizitează ipoteze specifice ridicate în discuția din Capitolul 4 și le testează direct, cu ajutorul unor diagnostice țintite – confirmând unele dintre ele și nuanțând altele.

Capitolul 6 (*Concluzii*) rezumă concluziile lucrării, discută limitările acestora și schițează posibile direcții de dezvoltare ulterioară.

Capitolul 2

Preliminarii

2.1 Proiectarea și Abordarea Cercetării

Pentru a răspunde la întrebarea formulată mai sus, am propus următoarea abordare: în loc să optimizez o singură arhitectură pentru un singur set de date, am realizat o comparație de-a lungul a două axe simultan.

Prima axă este **paradigma**: cele șapte modele evaluate (§2.2) au fost alese special pentru a acoperi patru moduri structural distincte de a reprezenta cum arată „normalul” – o **graniță** trasată într-un spațiu indus de un kernel (One-Class SVM), o **partiționare** a spațiului de caracteristici prin diviziuni aleatoare (Isolation Forest), un **bottleneck de compresie** învățat, care trebuie să renunțe la orice, cu excepția structurii dominante a datelor normale (cele trei autoencodere bazate pe reconstrucție), și un model **predictiv** al modului în care datele normale evoluează pas cu pas (LSTM predictiv, Transformer cauzal). Fiecare paradigmă codifică o presupunere implicită diferită despre ce anume face un punct anormal, ceea ce ridică întrebarea centrală a acestei lucrări: care presupunere se potrivește cărui tip de anomalie.

A doua axă este **setul de date**: Credit Card Fraud, Yahoo Webscope S5 și CIC-

IDS2017 (§2.3) au fost alese pentru a acoperi un spectru de structură temporală – de la un set de date pur tabelar, fără o ordonare semnificativă între rânduri, trecând prin serii de timp cu adevărat secvențiale, până la date de flux de rețea care admit atât o perspectivă plată, cât și una temporală asupra acelorași evenimente. Rularea fiecărei paradigme de model pe fiecare punct al acestui spectru este ceea ce face posibilă atribuirea unui rezultat dat *paradigmei*.

2.1.1 Ipoteze și Comportament Așteptat

Pe baza acestor două axe, anticipez două tipare în rezultate.

În primul rând, mă aștept ca paradigmele care oferă modelului o structură temporală inherentă a datelor – prin construcție, nu doar prin faptul că le sunt oferite ferestre de intrare – să performeze în general mai bine pe seturile de date cu adevărat secvențiale (Yahoo S5 și, într-o măsură mai mică, varianta temporală a CIC-IDS2017), comparativ cu paradigmele care tratează fiecare eșantion ca independent.

În al doilea rând, mă aștept ca diferite paradigme să fie mai bine adaptate la a detecta diferite *tipuri* de anomalii, chiar în cadrul acelui set de date. Dacă această presupunere este corectă, atunci compunerea mai multor paradigme complementare într-un ansamblu ar trebui să ducă la o îmbunătățire a performanței față de oricare model individual, atâta timp cât modelele componente exploatează relații diferite din date, și nu doar variații ale aceleiași relații. Această a doua ipoteză este testată direct atât prin analiza pe tip de atac din §4 (în special pentru CIC-IDS2017, unde este disponibilă eticheta tipului de atac), cât și prin studiul de caz al modelului ansamblu din §5.

În următoarele secțiuni vor fi prezentate detaliile algoritmilor și seturilor de date alese.

2.2 Algoritmi de Detecție a Anomaliilor

2.2.1 One-Class SVM

One-Class SVM (formularea ν -SVM) învață o graniță în jurul datelor de antrenare, într-un spațiu de caracteristici (de obicei infinit-dimensional) indus de un kernel. Trei hiperparametri, fixați identic pe toate cele trei seturi de date (`kernel='rbf'`, `nu=0.01`, `gamma='scale'` – vezi §1), determină împreună forma acestei granițe:

- `kernel='rbf'` fixează spațiul de caracteristici propriu-zis: fiecare punct de antrenare generează o **clopot gaussian** neted în jurul său, iar granița finală este conturul închis format prin suprapunerea acestor clopote în jurul zonelor dense de date.
- `gamma='scale'` stabilește cât de largă este fiecare clopot gaussian, deci cât de strâns se poate curba granița în jurul structurii locale a datelor – o valoare mai mare permite o graniță mai strânsă, dar cu risc mai mare ca modelul să învețe zgomotul.

- `nu=0.01` controlează cât de strâns este forțată granița să încadreze datele de antrenare, limitând la $\sim 1\%$ fracția de puncte benigne pe care modelul are voie să le lase în afara ei.

La testare, modelul returnează distanța semnată față de granița învățată (pozitivă în interior, negativă în exterior); aceasta este negată astfel încât, în concordanță cu toate celelalte modele din acest proiect, *scor mai mare = mai anormal*.

2.2.2 Isolation Forest

Isolation Forest construiește un ansamblu de arbori (numărul lor dat de parametrul `n_estimators`) care împart fiecare recursiv spațiul de caracteristici pe baza unor caracteristici și puncte de divizare alese aleator, și folosește lungimea medie a căii necesare pentru a izola un punct (în propria sa frunză), calculată pe toți arborii, drept semnal de anomalie. Deoarece diviziunile sunt aleatoare, punctele ușor de separat de restul datelor – adică anomaliile, care tind să se afle în regiuni rare, de densitate scăzută – sunt izolate în mai puține diviziuni, obținând astfel o lungime medie a căii mai mică decât punctele benigne tipice. Modelul returnează lungimea medie negată a căii, care este negată încă o dată aici, astfel încât un scor mai mare să însemne o cale mai scurtă (adică mai anormal), păstrând convenția consistentă cu toate celelalte modele. Parametrul `contamination` este lăsat la valoarea implicită, `'auto'`: acest parametru ar seta în mod normal pragul intern de decizie al modelului pe baza unei fracții presupuse de anomalii în datele de antrenare, dar cum datele de antrenare aici sunt exclusiv benigne (nicio contaminare de luat în calcul), iar pragul de operare este ales extern prin baleierea curbei PR din §3.2, `contamination` nu are niciun efect asupra rezultatelor raportate – contează doar scorul de anomalie continuu.

2.2.3 Autoencoder Feedforward (MLP)

Autoencoder Feedforward este bazat pe reconstrucție: o pereche simetrică encoder/decoder de straturi complet conectate **Linear + ReLU** (cu dropout între straturile ascunse) comprimă vectorul de caracteristici de intrare printr-un bottleneck de dimensionalitate strict redusă, apoi îl extinde înapoi la dimensiunea originală, fiind antrenat integral pentru a minimiza eroarea medie pătratică de reconstrucție față de propria intrare. Deoarece bottleneck-ul este mai îngust decât intrarea, rețeaua nu poate învăța pur și simplu funcția identitate; este forțată să învețe o reprezentare comprimată care surprinde structura dominantă specifică datelor *benigne*. La testare, scorul de anomalie este eroarea medie pătratică per eșantion dintre intrare și reconstrucția sa: eșantioanele benigne, care seamănă cu distribuția de antrenare în jurul căreia a fost modelat bottleneck-ul, se reconstruiesc cu eroare mică, în timp ce eșantioanele anormale – pe care bottleneck-ul nu a fost niciodată antrenat să le reprezinte – se reconstruiesc slab.

2.2.4 Autoencoder LSTM

Autoencoder LSTM aplică același principiu de reconstrucție ca Autoencoder Feed-forward, dar pentru intrări sub formă de secvențe: un strat LSTM funcționează ca encoder, consumând întreaga secvență de intrare și păstrând doar starea ascunsă finală (și starea de celulă) ca un singur vector de dimensiune fixă, rezumând întreaga secvență. Această stare ascunsă finală este apoi repetată identic la fiecare pas de timp, ca intrare pentru un al doilea strat LSTM care acționează ca decoder, ale cărui ieșiri per pas de timp sunt proiectate înapoi la dimensionalitatea originală a caracteristicilor printr-un strat liniar comun. Scorul de anomalie este, din nou, eroarea medie pătratică per eșantion, calculată pe toți pașii de timp și toate caracteristicile simultan, între secvența de intrare și reconstrucția sa.

2.2.5 LSTM Predictiv

LSTM Predictiv folosește același bloc recurent (un singur strat LSTM) ca Autoencoder LSTM de mai sus, dar cu un obiectiv de antrenare diferit: este antrenat să prezică pasul $t + 1$ din pașii $0..t$, simultan la fiecare poziție din secvență – modelul nu vede niciodată valoarea pe care este pus să o prezică. Din punct de vedere arhitectural, aceasta înseamnă că starea ascunsă a LSTM-ului la fiecare pas de timp este proiectată direct într-o predicție a pasului următor printr-un strat de ieșire liniar, fără separare encoder/decoder și fără vector de tip bottleneck: modelul consumă secvența o singură dată, iar fiecare stare ascunsă de-a lungul parcursului este folosită, nu doar cea finală. Scorul de anomalie este eroarea medie pătratică dintre valorile prezise și cele reale ale pasului următor, de-a lungul secvenței.

2.2.6 Autoencodere Bazate pe Transformer

Sunt folosite două variante de transformer, ambele înlocuind coloana vertebrală recurentă a Autoencoder LSTM cu un **TransformerEncoder** multi-strat (blocuri de auto-atenție + feedforward, codificare pozițională sinusoidală), diferind însă în modul în care este impusă constrângerea de bottleneck (reconstrucție) sau de cauzalitate (predicție):

- **Bottleneck Transformer Autoencoder** – bazat pe reconstrucție. Fiecare caracteristică/pas de timp de intrare este proiectat(ă) într-un token de dimensiune D_MODEL și trecut(ă) prin encoderul transformer cu auto-atenție completă (non-cauzală), astfel încât fiecare poziție poate atenționa liber orice altă poziție. Reprezentările rezultate per poziție sunt apoi *mediate global* (*global-average-pooled*) de-a lungul secvenței într-un singur vector de dimensiune D_MODEL , comprimat printr-un strat liniar la un $LATENT_DIM$ strict mai mic, apoi extins înapoi printr-un al doilea

strat liniar la forma completă a secvenței aplatizate. Acest pas de comprimare-extindere este bottleneck-ul explicit de informație (analog stării ascunse finale din Autoencoder LSTM) care împiedică mecanismul de atenție să copieze pur și simplu intrarea în ieșire. Scor de anomalie: MSE per eșantion între intrare și reconstrucție, pe toate pozițiile deodată – întreaga secvență reconstruită este produsă printr-o singură trecere prin rețea, nu pas cu pas ca la un decoder recurent.

- **Causal Transformer** – predictiv-temporal, echivalentul bazat pe atenție al LSTM Predictiv. Este folosită aceeași structură de encoder, dar cu o mască de atenție cauzală (triunghiulară superior) aplicată, astfel încât poziția t poate atenționa doar pozițiile $0..t$; modelul este antrenat cu același obiectiv de predicție a pasului următor ca LSTM Predictiv, fără bottleneck de comprimare. Scor de anomalie: MSE per eșantion între valorile prezise și cele reale ale pasului următor.

Ambele variante folosesc deliberat un singur **TransformerEncoder**, în loc de structura completă encoder-decoder întâlnită frecvent în literatura despre transformere pentru serii de timp (de exemplu, encoderul cu atenție ProbSparse al Informer, cuplat cu un decoder generativ, sau proiectele bazate pe reconstrucție care cuplează un encoder Transformer cu un decoder convoluțional separat). Costul de antrenare și inferență crește odată cu numărul de straturi de atenție, pe toate seturile de date și modelele, iar un design de tip encoder-only păstrează acest cost, împreună cu numărul de parametri antrenabili, la aproximativ jumătate din ce ar necesita un model echivalent encoder-decoder, fără a sacrifica abilitatea de a face fie comprimare-și-reconstrucție, fie predicție cauzală.

Nu există o implementare Causal Transformer pentru setul de date credit card (§4).

2.3 Seturile de Date

2.3.1 Credit Card Fraud Detection (Setul de Date A)

Setul de date Kaggle Credit Card Fraud Detection conține 284,807 tranzacții bancare, dintre care 492 (0.172%) sunt frauduloase. Cele 30 de caracteristici prezente sunt: **Time** (secunde scurse de la prima tranzacție din set), 28 de caracteristici transformate prin PCA V1–V28 (anonimizate din motive de confidențialitate, deja decorelate și aproximativ zero-medie prin construcție), și **Amount** (valoarea brută a tranzacției, puternic asimetrică spre dreapta). **Time** reprezintă secunde scurse, nu o axă temporală structurată – nu există periodicitate, tendință sau noțiune de sesiune de exploatat. Acest lucru face din Setul de Date A cazul de control deliberat, „fără ordine temporală reală”.

Setul de date, împreună cu analiza exploratorie a comunității, este disponibil pe Kaggle.¹ Este disponibil și un notebook exploratoriu suplimentar, dedicat special gestionării

¹<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

datelor dezechilibrate și detecției anomaliilor pe acest set de date.², care a servit drept inspirație pentru o parte din scripturile acestui proiect.

2.3.2 Yahoo Webscope S5 (Setul de Date B)

Yahoo Webscope S5 constă din patru familii de benchmark (A1–A4), fiecare o colecție de mai multe serii de timp univariate independente, cu anomalii etichetate, dar fiecare construită pentru a izola un tip diferit de anomalie:

Benchmark	Serii	Origine	Tip de anomalie	Rata de anomalii
A1	67	Metrice reale de server	Valori punctuale aberante	1.76%
A2	100	Sintetic, o singură componentă periodică + tendință	Valori punctuale aberante	0.33%
A3	100	Sintetic, 3 componente de frecvență în interacțiune	Contextuală (deviație de fază)	0.56%
A4	100	Sintetic, aceeași structură, fără sezonality	Schimbări de nivel	0.50%

A1 este real și zgomotos (cicluri zilnice/săptămânale dezordonate); A2–A4 sunt sintetice și construite special astfel încât tipul de anomalie să fie controlat independent de forma seriei de bază. Acest lucru este ceea ce permite capitolului de Rezultate să atribuie diferențele de performanță *tipului de anomalie* (punctuală vs. contextuală vs. schimbare de nivel), și nu unor diferențe confundante între seturile de date.

2.3.3 CIC-IDS2017 (Setul de Date C)

CIC-IDS2017 (Canadian Institute for Cybersecurity) este un benchmark de detecție a intruziunilor în rețea, captat pe parcursul a cinci zile: ~2.8 milioane de fluxuri de rețea, fiecare descris de ~80 de caracteristici statistice extrase prin CICFlowMeter (durată, numărul și rata pachetelor/octeților, statistici ale timpului dintre sosiri, numărul de flag-uri TCP, dimensiuni de fereastră etc.). Luni este trafic 100% benign; 14 tipuri distincte de atacuri (variante DoS/DDoS, PortScan, forță brută, atacuri web, Bot, Infiltration, Heartbleed) sunt injectate de marți până vineri.

Se știe că distribuția brută/Kaggle a acestui set de date este substanțial coruptă (Engelen et al., WTMC 2021): aproximativ 26% din fluxuri sunt artefacte de tip „appendix” TCP, care moștenesc eticheta de atac a fluxului părinte fără a conține trafic de atac real, o mare parte din fluxurile etichetate nominal ca atac nu au încărcătură utilă reală (etichetate doar formal, nu pe bază de comportament), iar întreaga clasă DoS Hulk corespunde unei simulări de atac defecte, care nu se execută niciodată efectiv împotriva serverului țintă. Sunt prezente și coloane de identificare (Flow ID, IP sursă/destinație, Timestamp) care, dacă sunt păstrate, permit unui model să „învețe pe scurtătură” după adresa IP sau fereastra de captură, în loc de comportamentul traficului. Acest proiect folosește

²<https://www.kaggle.com/code/janiobachmann/credit-fraud-dealing-with-imbalanced-datasets>

versiunea curățată și reextrasă a setului de date de la Engelen et al., în locul CSV-urilor originale Kaggle/UNB, și elimină explicit coloanele de identificare (vezi §3.1).

Fluxurile CIC-IDS2017 sunt agregate statistice la nivel de sesiune. Acest lucru face din acest set de date un caz de tranziție perfect: admite o perspectivă validă de tip plat/tabelar (un flux = un eșantion, analog setului de Date A), o perspectivă temporală validă (fluxuri grupate și ordonate per gazdă, analog setului de Date B).

Capitolul 3

Methods and Experimental Setup

Hiperparametrii și alegerile legate de antrenare, per model (optimizator, număr de epoci, lățimi ale straturilor și toate celelalte detalii de arhitectură, câte un tabel per model pentru cele patru variante de set de date/set de caracteristici), sunt prezentate în Anexa 1.

3.1 Preprocesare

Două principii se aplică la toate cele trei pipeline-uri: În primul rând, fiecare model este antrenat exclusiv pe eșantioane benigne/normale (*novelty detection*): partiția de antrenare nu conține niciodată o anomalie, iar toate scalarele/normalizatoarele sunt *potrivite (fit)* doar pe partiția de antrenare, apoi aplicate neschimbate pe partiția de testare – astfel evităm probleme de data leakage. În al doilea rând, partiția de testare conține întotdeauna ambele clase, evaluate cu protocolul descris în §3.2.

3.1.1 Preprocesare Tabelară (Credit Card)

1. **Time** este eliminat (nu reprezintă informație semnificativă, întrucât este doar timpul de la prima tranzacție din setul de date).
2. Rândurile sunt împărțite pe clase: toate rândurile benigne (**Class=0**) sunt reunite, apoi împărțite 80/20 în antrenare și test-benign. Setul de antrenare este partiția de 80%, conținând doar rânduri benigne (~227,451 rânduri).
3. Setul de testare este format din restul de 20% rânduri benigne, plus *toate* cele 492 de rânduri de fraudă.
4. Scalarea este asimetrică, pe grupuri de caracteristici: **V1--V28** trec **nescalate** implicit (**V_SCALING = 'none'** – întrucât acestea sunt deja caracteristici PCA), cu

'standard' încercat de asemenea în unele rulări, fără o diferență reală între modele. `Amount` este scalat independent cu un `RobustScaler` (bazat pe mediană/IQR), ales pentru robustețea sa la coada dreaptă accentuată, urmând același raționament ca notebook-ul exploratoriu citat în §2.

3.1.2 Crearea de Ferestre pentru Serii de Timp (Yahoo S5)

1. Fiecare serie este împărțită 70/30 de-a lungul axei temporale (`TRAIN_FRAC=0.70`) – o împărțire temporală și nu o amestecare aleatoare. Nu vrem ca vreo informație din viitor să se scurgă în antrenare.
2. Un `MinMaxScaler` este potrivit *per serie*, folosind doar punctele neanormale din partiția de antrenare a acelei serii, apoi aplicat atât partiției de antrenare, cât și celei de testare ale aceleiași serii.
3. Se aplică o fereastră glisantă de lungime `WINDOW=64`: pas 4 pentru ferestrele de antrenare (subeșantionate, păstrându-se pentru antrenare doar ferestrele complet normale) și pas 1 pentru ferestrele de testare (dens – este evaluată fiecare poziție posibilă a ferestrei).
4. Eticheta unei ferestre urmează regula „oricare”: pozitivă dacă cel puțin unul dintre cei 64 de pași de timp este anormal. Aceasta este totuși o sursă de „estompare” a etichetelor – de exemplu, pe A1, o rată de anomalii la nivel de punct de 1.76% devine o rată la nivel de fereastră de $\sim 17\text{--}18\%$, întrucât un singur punct anormal contaminează până la 64 de ferestre suprapuse. O fereastră mai mică ar reduce această estompare, lucru care ar putea merita încercat în viitor.
5. Ferestrele din toate seriile unui benchmark (reamintesc că fiecare benchmark este alcătuit din mai multe serii univariate) sunt reunite în array-uri unice.

Comparație cu DeepAnt. DeepAnt (Munir et al., 2019) a fost folosit ca referință pentru evaluarea unui predictor neural al pasului următor pe Yahoo S5, deși mai multe alegeri metodologice diferă de cele ale acestui proiect:

- **Predicție many-to-one, nu many-to-many.** Predictorul propus de DeepAnt este o rețea convoluțională; varianta LSTM raportată de aceștia ca referință de comparație (două straturi suprapuse, de câte 32 de unități) este mai îngustă și mai puțin adâncă decât LSTM-ul de 64 de unități folosit aici pentru LSTM Predictiv. Mai important, ambele prezic doar un singur pas de timp următor dintr-o fereastră (many-to-one), în loc de fiecare pas $t+1$ din $0..t$ simultan, așa cum fac modelele predictive ale acestui proiect (§2).

- **Antrenare per serie vs. antrenare reunită.** DeepAnt antrenează și evaluează un model separat pentru fiecare serie de timp individuală (o împărțire 40/60 antrenare/test per serie, scorul F calculat per serie și apoi mediat per sub-benchmark), ceea ce permite fiecărui model să se specializeze pe o singură frecvență. Acest proiect reunește în schimb ferestrele din toate cele ~ 100 de serii ale unui benchmark într-un singur model comun, antrenat pe o împărțire 70/30 – în principal din motive de simplitate a implementării.

3.1.3 Preprocesarea Fluxurilor de Rețea (CIC-IDS2017)

Există trei implementări pentru acest set de date, toate având la bază aceiași pași de curățare (eliminarea `Flow ID/Src IP/Dst IP/Timestamp/Src Port`; codificare one-hot pentru `Protocol`; înlocuirea `Inf` cu `NaN`; imputare cu mediana, calculată pe setul de antrenare; eliminarea coloanelor cu varianță zero, măsurată pe setul de antrenare; `MinMaxScaler` potrivit doar pe setul de antrenare), dar diferind prin unitatea de observație:

Plat Un rând = un flux. Setul de antrenare este format din toate fluxurile benigne din ziua de luni (371,749 rânduri); setul de testare conține toate fluxurile benigne de marți-vineri, plus toate fluxurile de atac reale (fluxurile -- **Attempted**, adică atacuri fără încărcătură observată, sunt excluse complet). Filtrarea coloanelor cu varianță zero pe datele de luni elimină cele trei coloane de flag URG, rămânând 77 din cele 80 de caracteristici originale.

Temporal Un rând = o fereastră de `WINDOW=16` fluxuri consecutive de la aceeași adresă IP sursă, sortate după timestamp. Această alegere de grupare – după IP sursă, nu după sesiunea individuală de flux (identificată printr-un 5-tuplu: IP sursă, IP destinație, port sursă, port destinație și protocol – cei cinci identificatori pe baza cărora `CICFlowMeter` definește o conexiune individuală) – corespunde cheii de grupare folosite și pentru calculul caracteristicilor de context de gazdă de mai jos. Ambele pipeline-uri tratează „o entitate” drept fluxul de activitate ordonat al unei gazde sursă, nu o conexiune 5-tuplu, întrucât comportamentul la nivel de gazdă (multe conexiuni scurte, în rafală) caracterizează atacuri precum `PortScan` sau `DoS`, mai degrabă decât structura internă a unei singure conexiuni. Împărțirea este pe zile (luni antrenare, marți-vineri testare), din același motiv ca la pipeline-ul plat.

Context de gazdă Aceeași curățare plată, per flux, ca mai sus, dar fiecare flux este suplimentat cu 9 caracteristici construite, calculate dintr-o fereastră temporală glisantă de 60 de secunde, per IP gazdă: numărul de fluxuri, numărul de porturi unice, rata de diversitate a porturilor, un indicator de port nou, precum și asimetria sursă/destinație a

fiecăreia. Acestea sunt calculate cauzal (doar din activitatea *anterioară* fluxului curent) și nu necesită etichete de atac, urmând adaptarea nesupervizată a lui Raskovalov et al. (2024). Numărul final de caracteristici: $77 + 9 = 86$. Această variantă a fost introdusă special pentru a rezolva o slăbiciune comună pipeline-urilor plat și temporal, pe clasa de atac PortScan (vezi §4).

3.2 Protocolul de Evaluare

La testare, fiecare model produce un scor scalar de anomalie per eșantion – niciodată o etichetă directă – iar această etichetă este obținută prin aplicarea unui prag asupra scorului, urmând procedura descrisă mai jos.

3.2.1 Metrici Raportate

Lista completă a metricilor raportate de fiecare rulare, precum și rezultatele complete pe toate aceste metrici pentru fiecare model și set de date, sunt prezentate în Anexa 2.

PR-AUC (Precizia Medie) este tratată drept metrica principală pentru compararea modelelor pe tot parcursul acestei lucrări, în locul F1 sau ROC-AUC. Aceasta urmează raționamentul lui Kim et al. (2022), care observă că prestabilirea unui prag de decizie fără acces la setul de testare nu este practică, și că metodele existente de detecție a anomaliilor în serii de timp raportează în schimb F1 la orice prag care se întâmplă să îl maximizeze – ceea ce face ca rezultatele raportate să fie dependente de prag prin construcție. În secțiunea de Discuții, aceștia concluzionează că „metrici suplimentare cu dependență redusă, precum AUROC sau aria de sub curba precizie-recall (AUPR), vor ajuta la o evaluare riguroasă”, ceea ce reprezintă justificarea adoptată aici pentru clasificarea modelelor după PR-AUC, în loc de F1 dependent de prag.

ROC-AUC - raportat de asemenea în capitolul următor, mai mult pentru completitudine / comparabilitate cu literatura anterioară.

Pentru CIC-IDS2017 în mod specific, este raportată suplimentar Precizia Medie defalcată pe tip de atac (tratând fiecare clasă de atac izolat, față de toate eșantioanele benigne), pentru a susține discuția per-tip-de-atac din §4.

3.2.2 Selecția Pragului

Se alege totuși un singur prag de decizie per model/set de date pentru a raporta metricile secundare, dependente de prag (F1, acuratețe echilibrată, matrice de confuzie) discutate mai sus: se baleiază toate pragurile de pe curba precizie-recall și se alege cel care maximizează F_β cu $\beta = 2$, ponderând recall-ul de două ori mai mult decât precizia. Valoarea exactă a lui β nu este reglată – orice $\beta > 1$ ar exprima aceeași preferință rezonabilă din punct de vedere operațional pentru un scenariu de detecție a anomaliilor/fraudelor,

în care o anomalie ratată este de obicei mai costisitoare decât o alarmă falsă; $\beta = 2$ a fost pur și simplu fixat ca o alegere reprezentativă.

Acesta este un prag de tip *oracol*: este selectat folosind chiar curba precizie-recall a setului de testare, care nu ar fi mereu disponibilă în realitate. Însă, am considerat acceptabil în acest caz întrucât obiectul de studiu este comparația relativă între modele/paradigme la cel mai bun punct de operare atins, iar PR-AUC, metrica principală, nu depinde deloc de selecția pragului. Așadar, cifrele F2/F1/acuratețe echilibrată la pragul oracol trebuie citite drept cifre secundare, ilustrative.

3.2.3 Fără Protocolul *Point Adjustment*

Multe lucrări despre detecția anomaliilor în serii de timp aplică *Ajustarea de Punct* (*Point Adjustment*, PA): dacă orice punct dintr-un segment contiguu de anomalie din datele de referință este semnalat, întregul segment este considerat ca fiind detectat. Kim et al. (2022) arată că acest lucru poate umfla F1 până aproape de 1 chiar și pentru scoruri aleatorii, atunci când segmentele de anomalie sunt lungi, și că F1 ajustat prin PA este aproape necorelat cu F1 neajustat (Pearson $r = -0.59$ pe setul de date SWaT, în studiul lor). Nicio ajustare PA nu este aplicată nicăieri în acest proiect: etichetele ferestrelor/fluxurilor sunt fixate la momentul preprocesării, iar predicțiile sunt evaluate exact așa cum rezultă din pragul aplicat, fără nicio ajustare retroactivă.

Capitolul 4

Experimental Results

4.1 Detectarea Fraudei cu Cardul de Credit (Setul de Date A)

4.1.1 Comparație Generală

Tabelul 4.1 sumarizează toate cele șase modele evaluate. Nu a fost folosită nicio variantă de Transformer Cauzal pentru acest set de date.

Toate cele șase modele ating ROC-AUC > 0.93 , dar AP variază între 0.519–0.711 $\rightarrow$ ROC-AUC este mult mai puțin sensibil la dezechilibrul de clasă față de AP.

4.1.2 Discuție

Nu există o secvență autentică în acest set de date. Spre deosebire de Yahoo S5 sau de variantele temporale/contextuale ale CIC-IDS2017, rândurile de tranzacții nu

Model	Paradigmă	ROC-AUC	Precizie Medie	F1 (fraudă)
MLP Autoencoder	Reconstrucție	0.9649	0.7105	0.6992
Isolation Forest	Partiționare	0.9587	0.6725	0.6070
LSTM Autoencoder	Reconstrucție (pseudo-secv.)	0.9476	0.6223	0.7362
OC-SVM	Frontieră	0.9354	0.5959	0.6454
Transformer Bottleneck AE	Reconstrucție (pseudo-secv.)	0.9583	0.5947	0.6137
Predictive LSTM	Predictiv (pseudo-secv.)	0.9581	0.5190	0.5244

Tabela 4.1: Detectarea fraudei cu cardul de credit – comparație de modele, clasificate după Precizia Medie.

au nicio ordonare temporală sau cauzală între caracteristici: **Time** este eliminată la pre-procesare (vezi secțiunea 3.1); pipeline-ul precede modelele secvențiale, adăugate ulterior exclusiv pentru comparabilitate între seturile de date. Cele trei arhitecturi ”temporale” evaluate aici (Predictive LSTM, LSTM AE, Transformer Bottleneck AE) nu văd o secvență reală – ele reformează fiecare rând cu 29 de caracteristici într-o secvență artificială de 29 de pași scalari ($(N, 29) \rightarrow (N, 29, 1)$), câte o caracteristică pe pas, în ordinea prezentă în fișierul .csv.

Impact asupra rezultatelor. Predictive LSTM este cel mai slab model (AP 0.519): obiectivul său de predicție a următorului pas presupune o relație cauzală între pași consecutivi, dar componentele PCA adiacente sunt ortogonale/decorelate prin construcție, deci nu există niciun semnal real pentru pasul următor de învățat – fapt reflectat în cel mai slab număr de fals-pozitive (FP=566). LSTM AE se descurcă comparativ bine (AP 0.622, cel mai bun F1 dintre toate modelele evaluate, 0.736, cel mai mic FP, 174) – o explicație ar fi faptul că encoderul său comprimă în continuare întreaga intrare într-o singură stare ascunsă sumară înainte de decodare, funcționând similar cu bottleneck-ul MLP AE, mai degrabă decât depinzând doar de ordine. Transformer Bottleneck AE (AP 0.595) se situează aproape de modelele clasice – similar ca mai sus, apare agregarea medie globală (global average pooling) într-un singur vector latent. MLP Autoencoder, singura arhitectură care tratează cele 29 de caracteristici ca pe un vector plat obișnuit, câștigă per total (AP 0.7105) – consistent cu faptul că nu există nicio secvență de exploatat.

Aceasta este imaginea în oglindă a constatării de la CIC-IDS2017, unde caracteristicile de context al gazdei (care codifică informații temporale/comportamentale *reale*) domină, și a constatării de la Yahoo, unde obiectivul predictiv câștigă în prezența unei structuri periodice autentice. Aici, impunerea unei structuri secvențiale acolo unde nu există niciuna pare să reprezinte un handicap.

Observații la nivel de paradigmă. Reconstrucția este cea mai puternică paradigmă per total, odată ce se ia în calcul handicapul pseudo-secvențial (MLP AE, LSTM AE și Transformer Bottleneck AE ocupă trei din primele patru locuri). Isolation Forest este

cel mai bun model din afara paradigmei de reconstrucție (AP 0.6725) și al doilea cel mai bun model per total – depășind notabil OC-SVM și două dintre cele trei arhitecturi temporale. OC-SVM (AP 0.5959) se situează la mijlocul clasamentului, cu al doilea cel mai mare număr de fals-pozitive după Predictive LSTM. Predictive LSTM este singurul reprezentant al paradigmei predictive pe acest set de date și cel mai slab model per total, din motivul structural discutat mai sus.

Sanity check. F1-ul cel mai bun dintre toate (0.7362) al LSTM AE, în ciuda unui AP mediocru (0.6223), este o reamintire a faptului că F1 este calculat la un singur prag oracol F2 și poate favoriza un punct de operare diferit de cel care maximizează aria de sub curba PR.

4.2 Yahoo Webscope S5 (Setul de Date B)

A2 este exclus din comparațiile de mai jos: la fel ca A1, conține doar anomalii punctuale, nu contextuale sau de tip schimbare-de-nivel, deci ar ridica aceeași provocare de putere discriminativă discutată pentru A1, fără a adăuga un tip nou de anomalie în comparație.

4.2.1 A1 – Anomalii Punctuale (Metrici Reale de Server)

Tabelul 4.2 sumarizează cele cinci modele evaluate pe A1. Causal Transformer și Transformer Bottleneck AE nu au fost rulate pe acest benchmark.

Model	Paradigmă	ROC-AUC	Precizie Medie
MLP Autoencoder	Reconstrucție	0.792	0.611
LSTM Autoencoder	Reconstrucție	0.794	0.600
OC-SVM	Frontieră	0.776	0.595
Predictive LSTM	Predictiv	0.789	0.588
Isolation Forest	Partiționare	0.765	0.511

Tabela 4.2: Yahoo S5 A1 (anomalii punctuale) – comparație de modele, clasificate după Precizia Medie.

Discuție. Toate cele cinci modele se grupează strâns în intervalul AP 0.51–0.61: MLP Autoencoder este cel mai bun (AP 0.611), iar Isolation Forest cel mai slab (AP 0.511), dar diferența este suficient de mică încât nicio arhitectură nu arată un avantaj structural aici. Anomaliile punctuale (vârfuri pe un singur pas temporal) nu necesită înțelegere temporală pentru a fi detectate.

4.2.2 A3 – Anomalii Contextuale/Sezoniere

Tabelul 4.3 sumarizează toate cele șapte modele evaluate pe A3.

Model	Paradigmă	ROC-AUC	Precizie Medie
Predictive LSTM	Predictiv	0.868	0.812
Transformer Bottleneck AE	Reconstrucție	0.762	0.663
Causal Transformer	Predictiv	0.738	0.623
MLP Autoencoder	Reconstrucție	0.765	0.507
LSTM Autoencoder	Reconstrucție	0.566	0.392
OC-SVM	Frontieră	0.584	0.376
Isolation Forest	Partiționare	0.511	0.331

Tabela 4.3: Yahoo S5 A3 (anomalii contextuale/sezoniere) – comparație de modele, clasificate după Precizia Medie.

Discuție. Rezultatele diverg puternic odată ce anomaliile devin contextuale – o valoare normală în termeni absoluți, dar greșită pentru faza sa în ciclul sezonier. Isolation Forest și OC-SVM se prăbușesc aproape de random (ROC-AUC 0.511 și, respectiv, 0.584). LSTM Autoencoder se prăbușește la fel de rău (ROC-AUC 0.566), în ciuda faptului că este un model secvențial – ceea ce arată că este vorba de o eșuare cauzată de blocajul de compresie (bottleneck), nu de o slăbiciune a arhitecturilor recurente în general. În aceeași ordine de idei, Transformer Bottleneck AE rămâne în urma celui mai bun model (AP 0.663), dar este de departe cea mai puternică arhitectură bazată pe *reconstrucție*, cu mult înaintea LSTM AE: auto-atenția globală asupra ferestrei recuperează probabil periodicitatea pe care blocajul LSTM AE o distruge, ceea ce probabil explică diferența. Predictive LSTM domină și este cel mai bun model pe orice benchmark Yahoo (AP 0.812, ROC-AUC 0.868): obiectivul său de predicție a pasului următor exploatează direct cât de predictibilă este o serie sezonieră, astfel încât o anomalie injectată produce un vârf clar în eroarea de predicție exact acolo unde violează traiectoria așteptată.

4.2.3 A4 – Anomalii de Tip Schimbare-de-Nivel

Tabelul 4.4 sumarizează toate cele șapte modele evaluate pe A4.

Discuție. Seriile modelează acum un sistem care experimentează schimbări structurale bruște. Fiecare model se degradează față de A3, indiferent de paradigmă. Un aspect important este că ferestrele din toate cele ~100 de serii sunt grupate laolaltă fără identitatea seriei, iar fiecare serie se situează la propriul nivel de bază – ar urma deci ca valoarea absolută a unei ferestre, luată singură, să nu poată distinge o fereastră autentică de după schimbare de o fereastră obișnuită dintr-o altă serie al cărei nivel de bază se întâmplă să

Model	Paradigmă	ROC-AUC	Precizie Medie
Predictive LSTM	Predictiv	0.745	0.505
Causal Transformer	Predictiv	0.695	0.436
MLP Autoencoder	Reconstrucție	0.719	0.402
Transformer Bottleneck AE	Reconstrucție	0.655	0.404
Isolation Forest	Partiționare	0.498	0.268
OC-SVM	Frontieră	0.519	0.265
LSTM Autoencoder	Reconstrucție	0.502	0.252

Tabela 4.4: Yahoo S5 A4 (anomalii de tip schimbare-de-nivel) – comparație de modele, clasificate după Precizia Medie.

fie similar – de unde și rezultatele mai slabe ale modelelor bazate pe distribuția agregată. Modelele clasice ating fundul complet sub această confuzie – Isolation Forest ajunge la ROC-AUC 0.498. Autoencoderele de reconstrucție (MLP AE, Transformer Bottleneck AE) sunt afectate din același motiv și nu reușesc să depășească Predictive LSTM aici, în ciuda faptului că o fereastră în formă de treaptă ar fi intuitiv mai ușor de semnalat prin eroarea de reconstrucție. LSTM Autoencoder se prăbușește complet (ROC-AUC 0.502), reproducând aceeași eșuare prin compresia encoderului observată la A3. Predictive LSTM încă conduce (AP 0.505, ROC-AUC 0.745), deși marja sa față de restul se îngustează puternic față de A3: deoarece își condiționează fiecare predicție pe istoricul recent propriu al unei serii, o schimbare este în continuare înregistrată ca o violare a continuității locale, indiferent de nivelul absolut la care apare, dar probabil generează un semnal de eroare susținut și mai zgomotos pe parcursul tranziției, în loc de vârful curat de deviație sezonieră pe care îl putea exploata mai ușor pe A3.

4.2.4 Verificare de Referință Trivială (Doar Yahoo)

Urmând Kim et al. (2022), care recomandă verificarea fiecărui model antrenat față de cel puțin o referință fără etichete (label-free) înainte de a atribui scorul acesteia unei învățări veritabile a reprezentărilor, modelele antrenate pe Yahoo S5 au fost de asemenea evaluate în raport cu o astfel de referință (varianta lor "Case 2"): se punctează fiecare fereastră după norma sa L2 brută $\|w\|_2$, fără niciun model învățat. Deoarece anomaliile punctuale în A1–A3 sunt vârfuri de amplitudine, o fereastră care conține una are magnitudine ridicată chiar și fără nicio învățare de reprezentări. Orice model antrenat care nu reușește să depășească această referință pe PR-AUC nu adaugă valoare față de amplitudinea brută a semnalului – această verificare este cea care confirmă că avantajele observate pe A3 sunt reale și nu un artefact al magnitudinii ferestrei.

Pe A3, referința însăși obține ROC-AUC 0.508 și AP 0.348 – random, așa cum era de așteptat pentru anomalii contextuale care sunt violări de fază, nu vârfuri de amplitudine. *Marja* pe care fiecare model antrenat o deține peste această referință este o cantitate

mai informativă decât AP-ul său brut: AP-ul propriu al referinței (0.348) se apropie îndeaproape de prior-ul de $\sim 33\%$ ferestre pozitive pe A3 (el însuși ridicat de estomparea etichetei la nivel de fereastră discutată în secțiunea 3.1), confirmând că referința este zgomot pur și că AP-ul brut al unui model este parțial o funcție a acestui prior umflat, nu doar a structurii învățate.

- **LSTM AE (+0.04 AP):** abia peste zgomot.
- **MLP AE (+0.16 AP):** captează implicit unele statistici privind forma ferestrei.
- **Transformere (+0.28–0.29 AP):** atenția globală asupra ferestrei de 64 de pași recuperează periodicitatea direct, o marjă mare și semnificativă.
- **Predictive LSTM (+0.47 AP):** depășește de peste două ori AP-ul referinței.

4.2.5 Discuție

Atât paradigma, cât și arhitectura contează. Rezultatele A3 formează o comparație de tip 2×2 între paradigmă (reconstrucție vs. predictiv) și arhitectură (LSTM vs. Transformer): reconstrucția obține 0.39 AP (LSTM AE) și 0.66 AP (Transformer Bottleneck), în timp ce paradigma predictivă obține 0.81 AP (Predictive LSTM) și 0.62 AP (Causal Transformer). Paradigma predictivă este necesară – ambele modele predictive depășesc ambele modele de reconstrucție – dar nu suficientă de una singură: diferența de 0.19 AP dintre Predictive LSTM și Causal Transformer, în ciuda unui obiectiv de antrenare identic și a acelorași date de antrenare, arată că arhitectura LSTM poartă un avantaj suplimentar pe date sezoniere. Pe A4, acest decalaj arhitectural se reduce la 0.07 AP – o singură tranziție bruscă nu necesită acumulare de fază, deci biasul inductiv secvențial al LSTM-ului contează mai puțin.

Rezumat pe tip de anomalie. Anomaliile punctuale (A1) lasă toate modelele într-o bandă îngustă de AP 0.51–0.61 – nu este necesară nicio înțelegere temporală pentru a semnala un vârf izolat. Anomaliile sezoniere (A3) sunt locul unde avantajul paradigmei predictive este cel mai clar, și unde alegerea arhitecturii contează cel mai mult în cadrul acelei paradigme. Punctele de schimbare (A4) degradează fiecare model, dar Predictive LSTM încă conduce, iar Causal Transformer încă depășește ambele modele de reconstrucție. Modelele clasice (Isolation Forest, OC-SVM) și LSTM Autoencoder se prăbușesc pe orice tip de anomalie care necesită discriminare contextuală, nu absolută (A3, A4), rămânând competitive doar pe A1.

4.3 CIC-IDS2017 (Setul de Date C)

4.3.1 Comparație Generală

Înainte de a compara modelele, Tabelul 4.5 detaliază pe clase fluxurile de atac agregate din setul de test flat/context, întrucât fiecare valoare AP raportată mai jos este calculată asupra întregii mulțimi agregate. Aceasta include **DoS Hulk** (158,469 fluxuri, la egalitate cu PortScan pentru cea mai mare clasă de atac). Totuși, Engelen et al. (WTMC 2021) arată că unealta de atac DoS Hulk folosită pentru a genera acest set de date este configurată greșit – setează antetul HTTP **Connection** pe **Close** în loc de **Keep-Alive**, astfel încât serverul web țintă termină conexiunea imediat, în loc să fie ținut ocupat, făcând atacul ineficient prin construcție. În consecință, DoS Hulk este exclus în mod deliberat din detalierea *pe clasă* din secțiunea 4.3.2. Această excludere se aplică doar raportării pe clasă, nu evaluării agregate: cele ~158 mii de fluxuri DoS Hulk rămân parte a clasei pozitive față de care este calculată fiecare metrică agregată din Tabelul 4.6.

Clasă de atac	n (fluxuri)	% din grupul de atacuri
PortScan	159,000	36.66%
DoS Hulk	158,469	36.54%
DDoS	95,000	21.91%
DoS GoldenEye	7,500	1.73%
DoS slowloris	4,000	0.92%
FTP-Patator	4,000	0.92%
SSH-Patator	3,000	0.69%
DoS Slowhttptest	1,700	0.39%
Bot	738	0.17%
Web Attack – Brute Force	151	0.03%
Infiltration	32	0.01%
Web Attack – XSS	27	0.01%
Web Attack – SQL Injection	12	<0.01%
Heartbleed	11	<0.01%

Tabela 4.5: CIC-IDS2017 – compoziția pe clase de atac a fluxurilor de atac agregate din setul de test flat/context. DoS Hulk contribuie la fiecare metrică agregată din Tabelul 4.6, dar este exclus din detalierea pe clasă din secțiunea 4.3.2 (vezi textul).

PortScan, DoS Hulk și DDoS reprezintă împreună 95.1% din toate fluxurile de atac din această mulțime – susținând afirmația, făcută repetat mai jos, că AP-ul agregat pe acest set de date este practic un proxy pentru performanța pe aceste trei clase. Celelalte zece clase rămase (toate sub 2% individual, șapte dintre ele sub 0.2%) au o influență limitată asupra clasamentului modelelor. Acesta este și motivul pentru care o detaliere pe atac este necesară (secțiunea 4.3.2 de mai jos).

Detalierea analoagă pentru setul de test cu ferestre al pipeline-ului temporal în Tabelul 3.1 din Anexă (Capitolul 3).

Model	Set de caracteristici	Paradigmă	ROC-AUC	Precizie Medie	Acuratețe Echilibrată	F1 (atac)
MLP Autoencoder	Context	Reconstrucție	0.9860	0.9487	0.9548	0.9020
OC-SVM	Context	Frontieră	0.9745	0.9396	0.9569	0.9229
Isolation Forest	Context	Partiționare	0.9785	0.9328	0.9270	0.8367
LSTM Autoencoder	Temporal	Reconstrucție	0.8976	0.7955	0.7846	0.6249
MLP Autoencoder	Flat	Reconstrucție	0.9189	0.7921	0.8022	0.6305
Isolation Forest	Flat	Partiționare	0.9049	0.7612	0.8228	0.6639
OC-SVM	Temporal	Frontieră	0.8629	0.7816	0.7600	0.5888
MLP Autoencoder	Temporal	Reconstrucție	0.8831	0.7509	0.7734	0.6125
Isolation Forest	Temporal	Partiționare	0.8438	0.7604	0.6841	0.5195
OC-SVM	Flat	Frontieră	0.7336	0.6845	0.6121	0.4644
Bottleneck Transformer AE	Temporal	Reconstrucție	0.8077	0.6539	0.7059	0.5423
Causal Transformer	Temporal	Predictiv	0.7099	0.5449	0.5520	0.4320
Predictive LSTM	Temporal	Predictiv	0.6126	0.4472	0.5475	0.4295

Tabela 4.6: CIC-IDS2017 – toate combinațiile model/set-de-caracteristici, clasificate după Precizia Medie.

Caracteristicile de context al gazdei domină. Cele trei modele augmentate cu context ocupă primele trei poziții la fiecare metrică, cu AP între 0.933–0.949. Diferența dintre cel mai bun model cu context (AE+Context, AP 0.949) și cel mai bun model fără context (MLP AE Flat, AP 0.792) depășește întreaga plajă acoperită de celelalte zece modele. Fiecare dintre cele trei arhitecturi de bază se îmbunătățește substanțial la adăugarea caracteristicilor de context: IForest câștigă +0.172 AP (0.761 $\rightarrow$ 0.933), MLP AE câștigă +0.157 (0.792 $\rightarrow$ 0.949), iar OC-SVM câștigă cel mai mult, +0.255 (0.685 $\rightarrow$ 0.940). Aceasta arată că uneori caracteristicile mai bune contează mai mult decât alegerea paradigmei.

Adăugarea ferestrelor temporale, singură, nu ajută. Pentru IForest și MLP AE, gruparea fluxurilor în ferestre ordonate după IP-ul sursă (secțiunea 3.1) fără adăugarea caracteristicilor de context nu oferă o îmbunătățire semnificativă față de vederea flat, și uneori chiar dăunează: IForest Flat (AP 0.761) $\approx$ IForest Temporal (AP 0.760), în timp ce AE Flat (AP 0.792) $>$ AE Temporal (AP 0.751). E un contrast direct cu câștigul din contextul gazdei de mai sus: ambele modificări adaugă informație despre *ordinea/activitatea fluxurilor în timp*, dar doar statisticile de context agregate se traduc într-un semnal utilizabil pentru cele două clase de atac (DDoS, PortScan) care domină setul de test ca volum. Ferestrele temporale poartă de fapt aceeași informație de activitate de bază, răspândită pe 16 fluxuri ordonate arbitrar, ceea ce reprezintă o reprezentare mai dificil de exploatat direct pentru aceste arhitecturi.

Reconstrucția depășește predicția. Printre modelele cu caracteristici temporale, arhitecturile bazate pe reconstrucție (LSTM AE, AE Temporal, OC-SVM Temporal) depășesc clar ambele modele bazate pe predicție (Predictive LSTM AP 0.447, Causal Transformer AP 0.545) – inversul constatării de la Yahoo A3 (capitolul 4). Sweep-urile de prag

ale ambelor modele predictive ajung la un punct de operare extrem ($FP > 1.1$ mil. din 1.28 mil. de fluxuri benigne), consistent cu ROC-AUC-ul lor scăzut (0.61 și 0.71).

4.3.2 Detaliere pe Tip de Atac

Tabelul 4.7 detaliază Precizia Medie pe clasă de atac pentru cele trei modele cu context, familia cea mai puternică per total. Dimensiunile claselor variază cu patru ordine de mărime (11 până la 159,000 eşantioane), fapt care influențează ce anume este detectabil în primul rând.

Clasă de atac	n	IForest+Ctx	OC-SVM+Ctx	AE+Ctx
DDoS	95,000	0.982	0.998	0.994
PortScan	159,000	0.601	0.820	0.842
DoS GoldenEye	7,500	0.655	0.215	0.220
DoS slowloris	4,000	0.190	0.012	0.025
DoS Slowhttptest	1,700	0.112	0.007	0.013
Heartbleed	11	0.093	0.001	0.000
Infiltration	32	AP 0.03–0.14, fără un câștigător clar la niciun model		
Bot	738	<0.01 la toate modelele, inclusiv variantele cu context		
FTP-Patator	4,000	<0.01 la toate modelele, inclusiv variantele cu context		
SSH-Patator	3,000	<0.01 la toate modelele, inclusiv variantele cu context		
Subtipuri Web Attack	12–151	AP aproape zero peste tot (Brute Force, SQL Injection, XSS)		

Tabela 4.7: CIC-IDS2017 – Precizia Medie pe clasă de atac pentru cele trei modele augmentate cu context.

Detectate constant. Două clase de atac ies în evidență ca fiind detectabile fiabil pe aproape orice set de caracteristici:

- **DDoS** este cea mai ușoară clasă, cu o marjă mare: toate modelele cu context obțin $AP > 0.98$ (OC-SVM+Context atinge 0.998), și chiar și modelele flat depășesc $AP > 0.77$. Rata foarte mare de pachete și dimensiunile mici ale pachetelor, caracteristice unei inundații, produc o valoare aberantă extremă atât în spațiul de caracteristici per-flux, cât și în cel de context al gazdei.
- **PortScan** este cea mai clară demonstrație a valorii caracteristicilor de context: fiecare model flat și temporal obține $AP > 0.07$ –0.28, în timp ce fiecare model cu context depășește 0.60, AE+Context atingând 0.842. Un singur flux de scanare scurt, către un port arbitrar, este nesemnificativ izolat, dar luând în considerare și numărul de porturi de destinație unice contactate de IP-ul sursă în ultimele 60 de secunde este ceea ce ajută probabil în mod special la acest atac. IForest+Context este comparativ mai slab aici (0.601).

Răspuns mixt la caracteristici de context pentru DoS la nivel de aplicație. Nu fiecare clasă de atac răspunde la caracteristicile de context în același mod – cele trei clase DoS la nivel de aplicație împart familia de context în loc să o favorizeze uniform:

- **DoS GoldenEye** arată o separare clară în cadrul familiei de context: IForest+Context atinge AP 0.655, în timp ce AE+Context și OC-SVM+Context se prăbușesc amândouă la ~ 0.22 .
- **DoS Slowhttptest și DoS slowloris** arată același tipar, mai atenuat: IForest+Context este singurul model cu context cu detecție semnificativă (AP 0.112 și 0.190), deși rămâne totuși mult sub AP-ul de 0.69/0.61 al IForest Temporal pe aceleași clase. Atacurile HTTP lente mențin multe conexiuni concurente de lungă durată către un singur port. Fereastra de context de 60 de secunde este o nepotrivire parțială aici, deoarece conexiunile Slowloris durează minute întregi, iar multe fluxuri cad în afara ei – ordonarea cronologică explicită per-gazdă a IForest Temporal oferă în continuare cea mai bună detecție pentru această familie de atacuri.

În mare parte nedetectabile, indiferent de setul de caracteristici. FTP-Patator, SSH-Patator, Bot și cele trei subtipuri Web Attack obțin toate AP < 0.01 la fiecare model, inclusiv cele cu context. Atacurile peste protocoale legitime și atacurile la nivel HTTP (injecție SQL, XSS) produc statistici de flux indistinctibile de utilizarea normală – detectarea lor ar necesita contoare de autentificare eșuată sau inspecția payload-ului, niciuna disponibilă într-un set de caracteristici de statistici de flux. Heartbleed (11 eșantioane) și Infiltration (32 de eșantioane) sunt prea mici pentru a trage concluzii solide, deși avantajul modest al IForest+Context pe Heartbleed (AP 0.093 față de aproape zero pentru celelalte două modele cu context) este consistent cu același avantaj de izolare aliniată-pe-axe observat la clasele DoS lente.

4.3.3 Discuție

Caracteristicile de context al gazdei sunt cea mai impactantă adăugare din acest studiu. Adăugarea a 9 statistici de activitate per-gazdă cu fereastră glisantă (secțiunea 3.1) ridică AP-ul de la 0.79 la 0.95 per ansamblu – un câștig mai mare decât alegerea paradigmei sau trecerea de la o vedere flat la una temporală asupra acelorași date – determinat în principal de rezolvarea punctului orb PortScan pe care îl aveau în comun toate modelele flat și temporale. Niciun model de context, luat singur, nu domină la fiecare tip de atac, deși AE+Context câștigă la AP-ul agregat, determinat de cele două clase cele mai mari.

Un ansamblu practic. Niciun model sau set de caracteristici, luat singur, nu acoperă fiecare clasă de atac din acest set de date. Combinarea AE+Context (puternic pe DDoS

și PortScan, cele două clase care domină setul de test ca volum) cu IForest Temporal (puternic pe clasele DoS lente și, cu titlu provizoriu, pe Heartbleed) ar acoperi cea mai largă gamă de clase de atac detectabile în cadrul acestui set de caracteristici. Această premisă, precum și un ansamblu cu scor combinat construit și evaluat pe baza ei, sunt testate direct în secțiunea 5.2: evidența celor două modele este confirmată ca fiind cu adevărat disjunctă, dar combinarea scorurilor lor se dovedește a schimba calitatea detecției PortScan pentru câștiguri pe clasele DoS lente, mai degrabă decât să le acopere pe amândouă gratuit.

4.4 Exploratory Data Analysis

This section collects the exploratory plots underlying the dataset-specific claims made in §3.1 and throughout this chapter.

4.4.1 Credit Card Fraud Detection

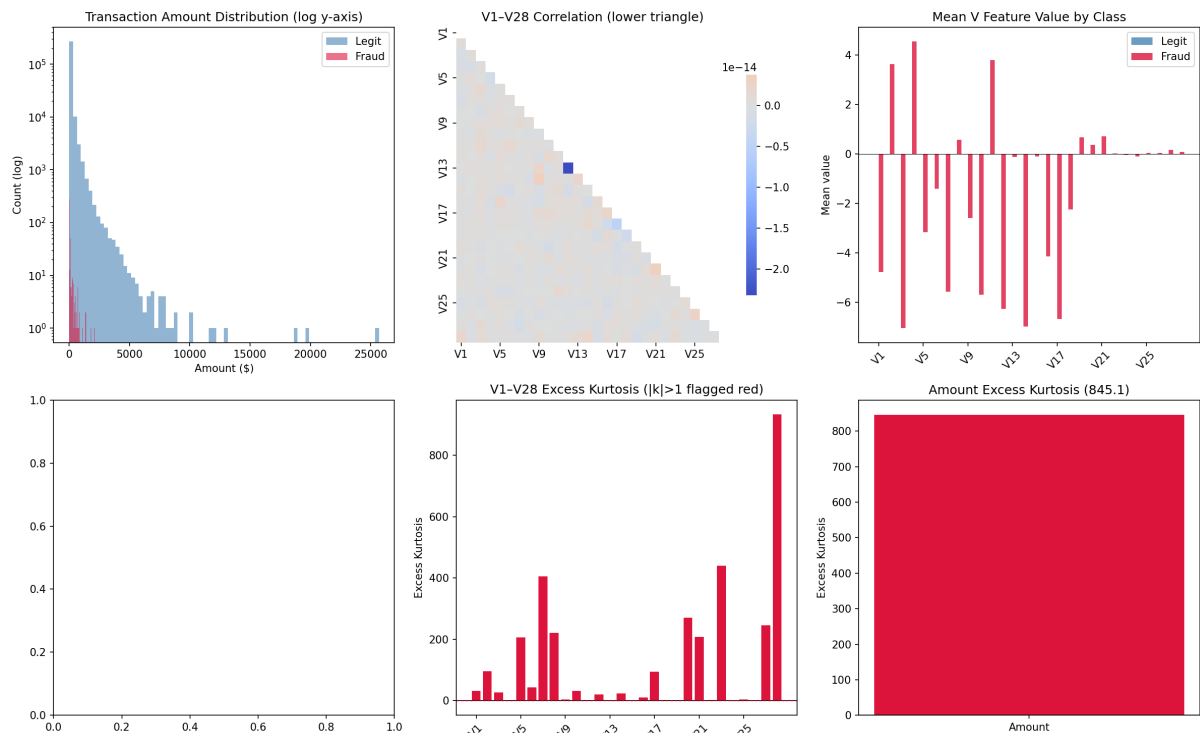


Figura 4.1: Credit Card Fraud Detection – transaction amount distribution, V1–V28 correlation structure, mean V-feature value by class, and excess kurtosis per feature (flagging the heavy-tailed PCA components discussed in the Datasets section).

4.4.2 Yahoo Webscope S5

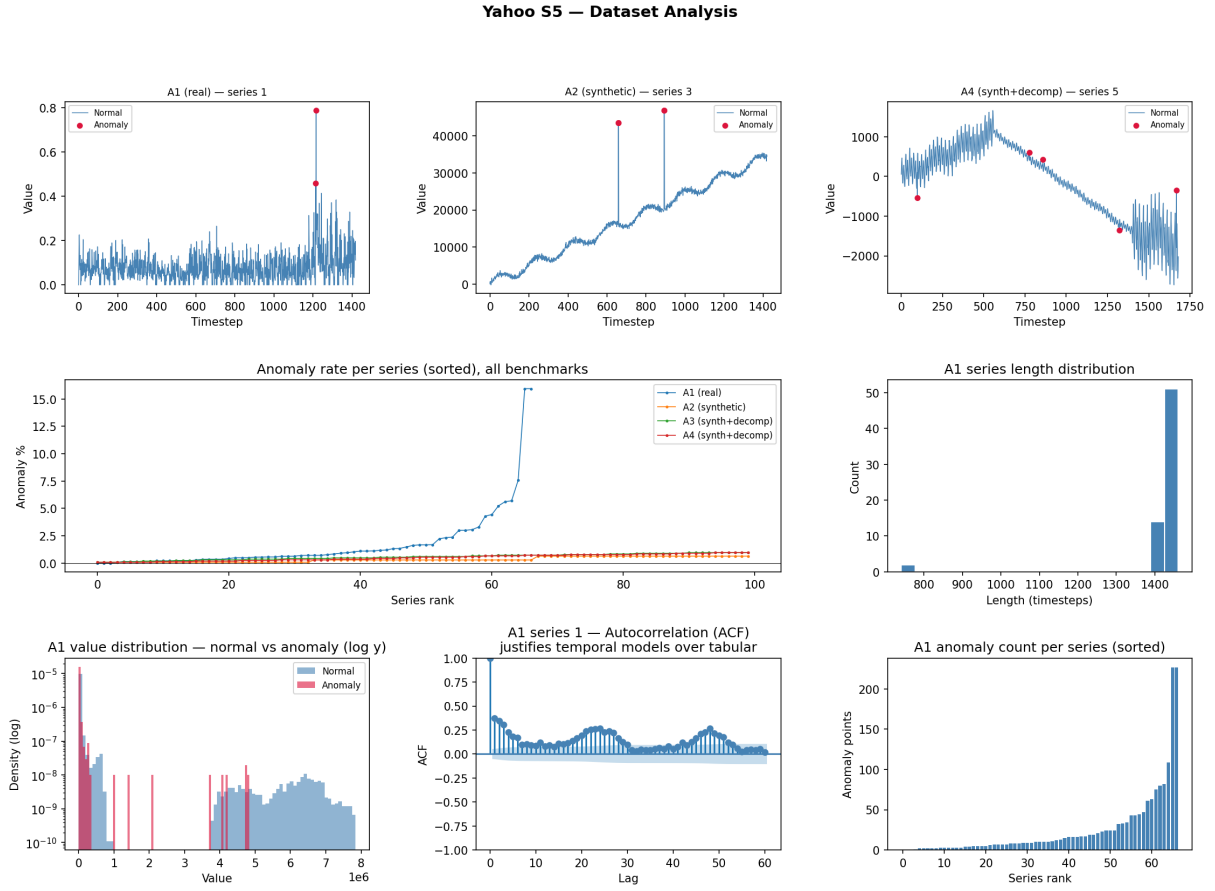


Figure 4.2: Yahoo S5 – example series per benchmark, per-series anomaly rate, A1 series length distribution, A1 value distribution for normal vs. anomalous points, A1 autocorrelation (motivating the use of temporal models over tabular ones), and A1 anomaly count per series.

4.4.3 CIC-IDS2017

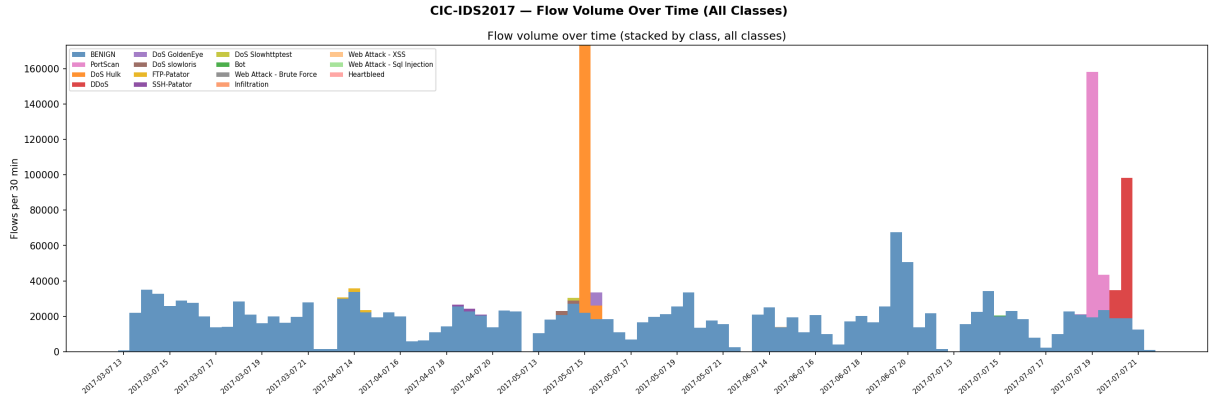


Figure 4.3: CIC-IDS2017 – flow volume over the capture week, stacked by class. BENIGN traffic dominates volume outside the concentrated attack windows visible on Tuesday–Friday.

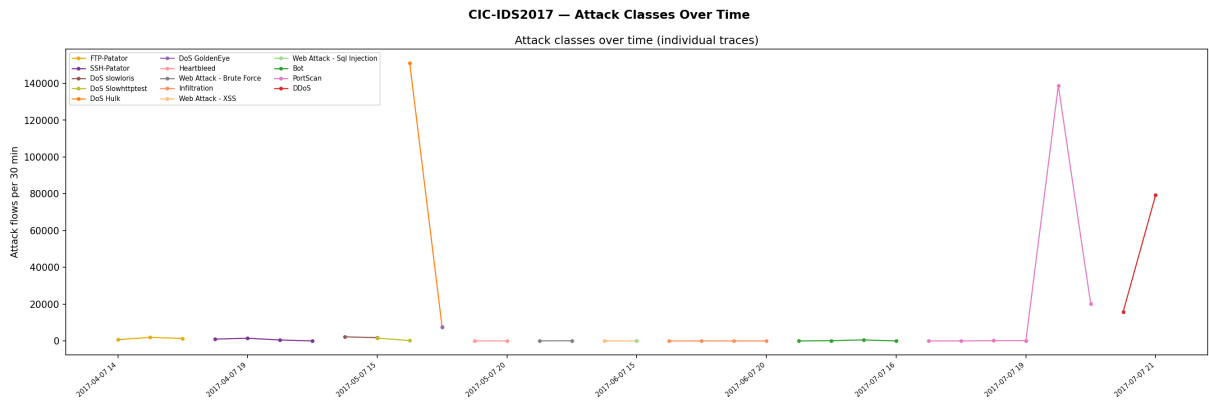


Figure 4.4: CIC-IDS2017 – attack classes over time, individual traces. Each attack occupies a short, distinct time window, confirming that attack classes do not overlap temporally within the capture.

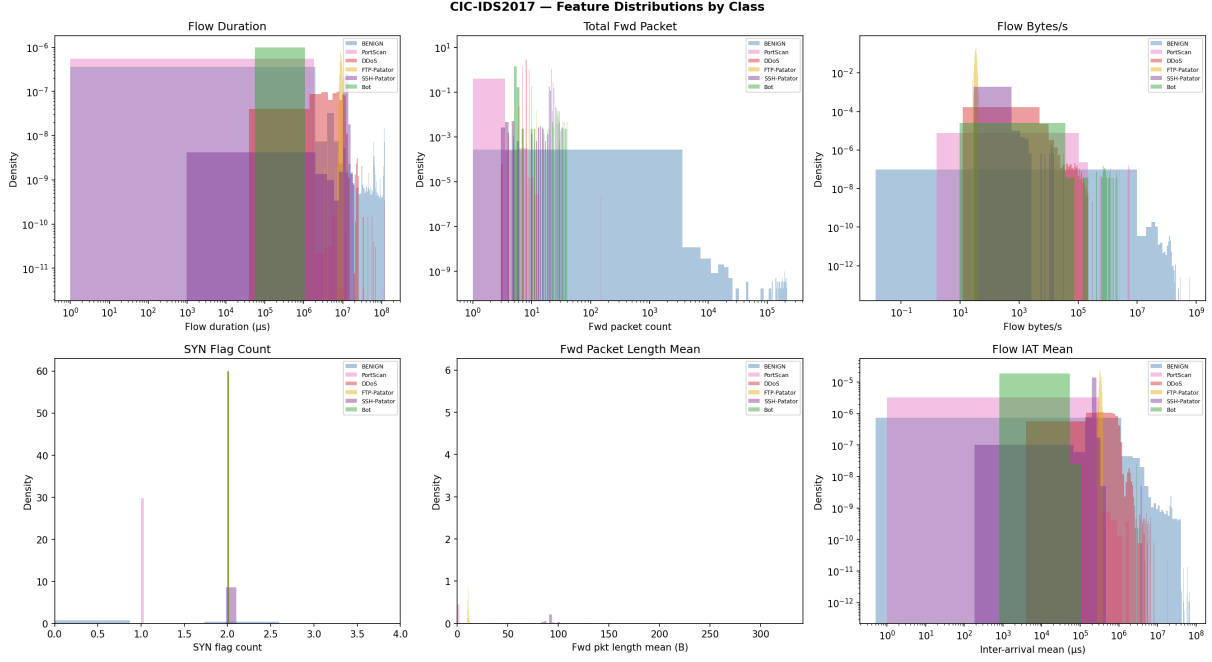


Figura 4.5: CIC-IDS2017 – per-class distributions of six flow-level features (flow duration, forward packet count, flow bytes/s, SYN flag count, forward packet length mean, inter-arrival mean) for BENIGN and five representative attack classes.

Capitolul 5

Explainability Case Studies

This chapter tests specific mechanistic claims made in Chapter 4 with targeted diagnostics, rather than relying on the aggregate metrics alone to support the mechanisms offered there.

5.1 Reconstruction-Error Localization on Yahoo A3

The A3 discussion in Chapter 4 attributes the Transformer Bottleneck Autoencoder’s lead over the LSTM Autoencoder (AP 0.663 vs. 0.392, ROC-AUC 0.813 vs. 0.566) to a mechanistic difference between the two bottlenecks: the LSTM Autoencoder compresses an entire 64-step window into a single final hidden state, which is hypothesized to destroy the window’s seasonal phase information and produce reconstruction error that is smeared diffusely across all timesteps; the Transformer’s global self-attention, by contrast, is hypothesized to preserve enough positional information to localize its reconstruction error sharply at the specific timestep where the window’s phase is violated. This is a plausible

mechanism, but as stated it is inferred from the aggregate metrics alone – neither model’s per-timestep error was actually inspected. This section tests it directly.

5.1.1 Initial Assumption

The hypothesis under test makes an explicit, checkable prediction: for anomalous A3 windows, the LSTM Autoencoder’s per-timestep squared reconstruction error should look approximately *uniform* across the window’s 64 positions, while the Transformer’s per-timestep error should show a sharp, localized peak at the position where the seasonal pattern is violated. If instead both models produce similarly shaped error profiles, the AP/ROC-AUC gap would need a different explanation than the one offered.

5.1.2 Qualitative Check: Per-Timestep Error Heatmaps

Both trained models were reconstructed on every anomalous A3 test window, and the per-timestep squared error was plotted as a heatmap (one row per window, one column per of the 64 timesteps), with a random subsample of 300 windows used to keep the rendered image legible. Figures 5.1 and 5.2 show the result.

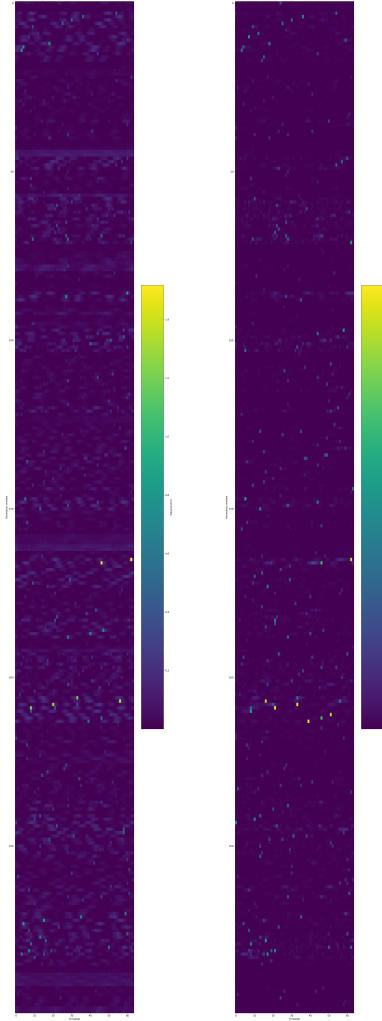


Figura 5.1: LSTM Autoencoder – per-timestep squared reconstruction error on 300 randomly sampled anomalous A3 windows (one row per window, left). Elevated error appears as sparse, punctate points rather than as a uniform band, but a visibly larger share of rows carry a low-level elevated baseline across most of their width compared to the Transformer.

Figura 5.2: Transformer Bottleneck Autoencoder – per-timestep squared reconstruction error on the same 300 anomalous A3 windows, same row order and colour scale convention as Figure 5.1 (right). Error is visibly more sparse and confined to isolated timesteps.

The visual comparison is broadly consistent with the initial assumption – the Transformer’s error concentrates into fewer, brighter points, while the LSTM Autoencoder’s error is more often spread thinly across a window’s full width – but a heatmap read by eye is a weak instrument here: each panel is normalized to its own colour scale, and the LSTM Autoencoder’s error values are roughly double the Transformer’s in magnitude throughout, which alone would make its background look more saturated regardless of how concentrated the error actually is. A qualitative read of Figures 5.1–5.2 therefore

suggests the phase-destruction mechanism but cannot confirm it.

5.1.3 Quantitative Verification: Gini Coefficient of Error Concentration

To separate genuine error concentration from an artifact of differing overall error magnitude, each anomalous window’s 64 per-timestep squared errors were summarized with a Gini coefficient – a standard inequality measure (originally used for income distributions) that is invariant to the absolute scale of its input. For a window’s 64 error values sorted ascending, a Gini of 0 means the error is spread perfectly evenly across every timestep, and a Gini of 1 means it is concentrated entirely at a single timestep; unlike a raw magnitude comparison, this metric isolates the *shape* of the error distribution within a window from its overall size.

For n per-timestep squared errors $x_1 \leq x_2 \leq \dots \leq x_n$ sorted ascending (here $n = 64$), the Gini coefficient is computed as

$$G = \frac{2 \sum_{i=1}^n i x_i}{n \sum_{i=1}^n x_i} - \frac{n+1}{n}. \quad (5.1)$$

Computed over the full population of 14,449 anomalous A3 windows, the two models separate clearly:

	Mean Gini	Median Gini
LSTM Autoencoder	0.566	0.606
Transformer Bottleneck Autoencoder	0.721	0.724

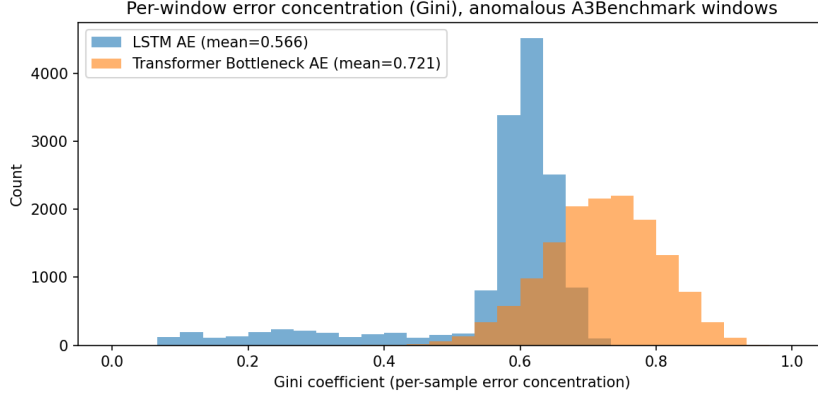


Figure 5.3: Distribution of per-window Gini coefficients across all 14,449 anomalous A3 windows. The Transformer’s distribution is unimodal and shifted toward high concentration (Gini > 0.6); the LSTM Autoencoder’s distribution is bimodal, with a majority clustered near the Transformer’s range but a distinct secondary population spread across low Gini values (error genuinely smeared across the window).

This confirms the initial assumption on average: the Transformer’s error is more concentrated than the LSTM Autoencoder’s across the full anomalous population, not merely in the handful of examples a heatmap happens to display. But the distribution in Figure 5.3 reveals a detail the aggregate means alone would miss – the LSTM Autoencoder’s Gini values are *bimodal*, not uniformly shifted down. A majority of its windows cluster in essentially the same high-concentration range as the Transformer’s (Gini ≈ 0.6 , comparable localization ability), while a distinct secondary population of windows sits at low Gini (< 0.4), where the error genuinely is smeared close to evenly across the full window. This indicates the phase-destruction mechanism is not a uniform property of every LSTM Autoencoder reconstruction, but a failure mode affecting a specific subset of windows – and it is this subpopulation, not a general degradation across all anomalous windows, that is responsible for dragging the LSTM Autoencoder’s aggregate AP and ROC-AUC below the Transformer’s, since on the remaining majority of windows the two models localize error comparably well.

5.2 Disjoint Feature Regimes: AE+Context vs. Temporal IForest

The CIC-IDS2017 Discussion in Chapter 4 proposes pairing AE+Context (strong on DDoS and PortScan) with Temporal IForest (strong on the slow-DoS classes and, tentatively, Heartbleed) into a practical ensemble, since the two operate on disjoint feature regimes – host-context features versus a 16-flow temporal window. That argument rests on two separate premises, one per model: that Temporal IForest’s score is genuinely in-

formed by the window’s temporal structure rather than by one flow’s static feature values sitting inside a wider, functionally inert vector; and that AE+Context’s score is genuinely informed by the host-context features unique to it, rather than by the same flow-level features Temporal IForest already has access to. Both are tested in turn below.

5.2.1 Temporal IForest: Initial Assumption

If Temporal IForest is genuinely exploiting cross-timestep structure, its per-window score should remain informative about the *last* flow’s own label – the flow that would be paired with AE+Context’s per-flow score in the ensemble – rather than only about whether *some* flow within the 16-flow window is an attack. Its SHAP attribution should also not collapse onto that single last flow; if it does, the window is decorative and the “disjoint feature regime” argument for the ensemble does not hold.

5.2.2 Temporal IForest: Score Validity Under Last-Flow Labeling

The originally showed per-class AP figures use an “any-of-16” window label: a window is positive if any of its 16 flows is an attack, tagged with the majority attack type. That is more lenient than the per-flow labeling AE+Context is scored against. Every existing test window was relabeled by its *last* flow’s own true label instead, and per-class AP recomputed against the unchanged, already-trained model and its existing scores.

	AP (any-of-16)	AP (last-flow)	<i>n</i> (last-flow)
DDoS	0.9518	0.9515	95,123
DoS GoldenEye	0.6278	0.6265	7,567
DoS Slowhttptest	0.6851	0.6849	1,742
DoS slowloris	0.6091	0.6077	4,001
Heartbleed	0.5927	0.2892	11

We see AP is essentially unchanged for all four target classes under the stricter labeling, confirming the score is informative about the last flow’s own status, not an artifact of the lenient window label. Heartbleed drops sharply, but on only 11 true flows – its entire test population – so it is excluded from the ensemble and treated as tentative, consistent with its hedged treatment in Chapter 4.

5.2.3 Temporal IForest: SHAP Attribution Across the Window

Temporal IForest is trained on flattened 16-flow windows, so TreeExplainer’s SHAP values on the flattened input decompose cleanly back into a (window, timestep, feature)

cube. For each of the four classes, SHAP was computed on 200 windows; attribution magnitude was summed over features per timestep to check for concentration on the final timestep, and the top contributing features were plotted across all 16 timesteps to check whether the same feature carries consistent-sign attribution throughout the window rather than only near the end.

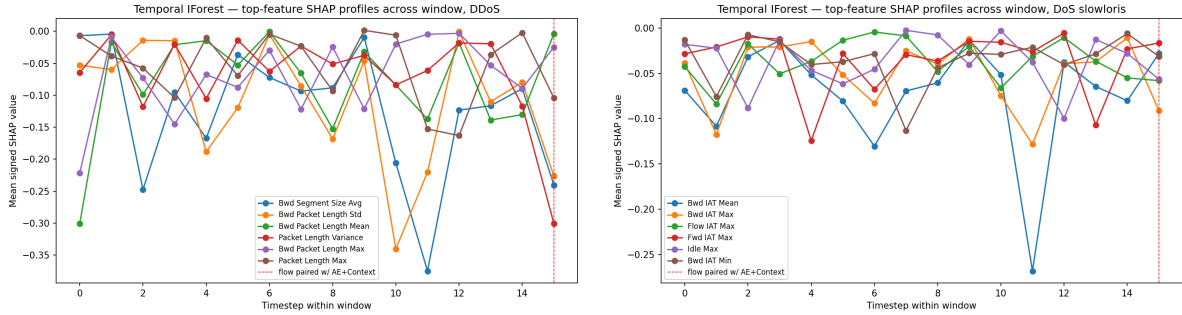


Figure 5.4: Temporal IForest – mean SHAP value per timestep for the six most-attributed features, DDoS windows (left) and DoS slowloris windows (right). In both cases attribution is negative (anomalous), recurs at consistent sign across most of the 16 timesteps, and does not concentrate at the final flow (dashed red line) – the flow paired with AE+Context’s score in the proposed ensemble.

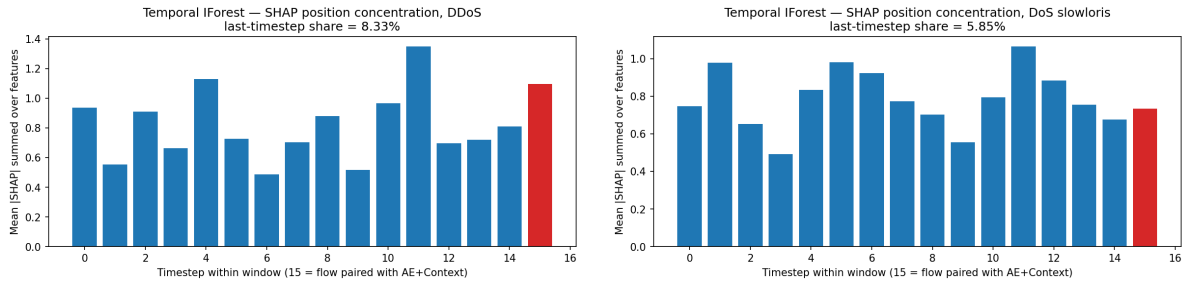


Figure 5.5: Temporal IForest – mean $|\text{SHAP}|$ summed over features, per timestep, DDoS windows (left) and DoS slowloris windows (right). The final timestep (red bar, 15) – the flow paired with AE+Context’s score in the proposed ensemble – carries no more attribution mass than any other position in the window, confirming the summary statistic reported below.

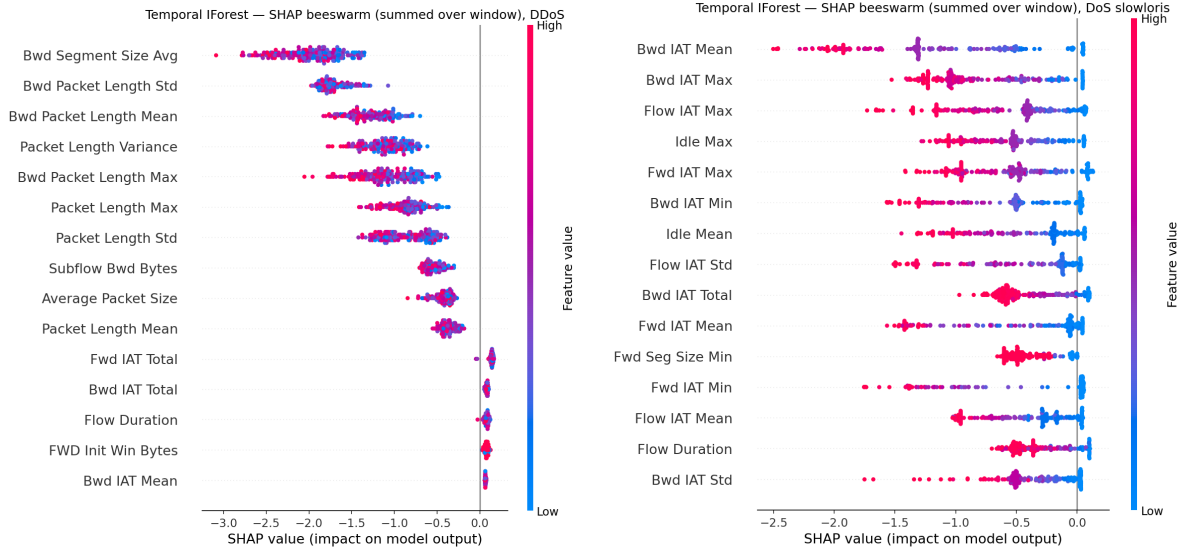


Figure 5.6: Temporal IForest – SHAP beeswarm per feature, DDoS windows (left) and DoS slowloris windows (right), collapsing the 16-timestep axis by summing each feature’s SHAP contribution across the window and coloring by that feature’s mean value across the window. DoS slowloris shows a clean monotonic relationship – high inter-arrival-time values drive strongly negative (anomalous) SHAP, low values push near zero – consistent with the slow, connection-holding behaviour the attack is named for. DDoS shows a noisier value-to-sign relationship for its backward-packet-length features, and a wider per-window spread than the mean-only view in Figure 5.4 suggests on its own.

The final timestep – the one paired with AE+Context in the ensemble – carries only 6–8% of total attribution mass across all four classes, close to the 6.25% uniform baseline for a 16-step window, rather than dominating it. The top features also match each attack’s known mechanism: DDoS and DoS GoldenEye attribution concentrates on backward-packet-length statistics (**Bwd Packet Length Std/Mean/Max**, **Packet Length Variance**), consistent with a volumetric flood; DoS Slowhttpptest and DoS slowloris attribution concentrates on inter-arrival-time statistics (**Fwd/Bwd IAT Std/Max**, **Idle Mean/Max**), consistent with the slow, connection-holding behaviour these attacks are named for. This confirms the initial assumption: Temporal IForest’s advantage on these classes reflects genuinely disjoint temporal evidence, not a relabeled copy of one flow’s static features – supporting the “disjoint feature regime” premise behind pairing it with AE+Context.

5.2.4 AE+Context: SHAP Dominated by Host-Context Features

The mirror premise is that AE+Context’s advantage on its own strong classes – DDoS and PortScan, per Chapter 4 – traces to the 9 `ctx_*` host-context features unique to its

feature set, rather than to the same flow-level features Temporal IForest already sees. If instead AE+Context’s attribution spreads across ordinary flow features, its complementarity with Temporal IForest would be coincidental rather than a genuinely disjoint evidence source.

SHAP was computed with `GradientExplainer` on AE+Context’s reconstruction error, on 200 samples each of PortScan and DDoS, and the 15 features with the highest mean $|\text{SHAP}|$ were ranked for each class.

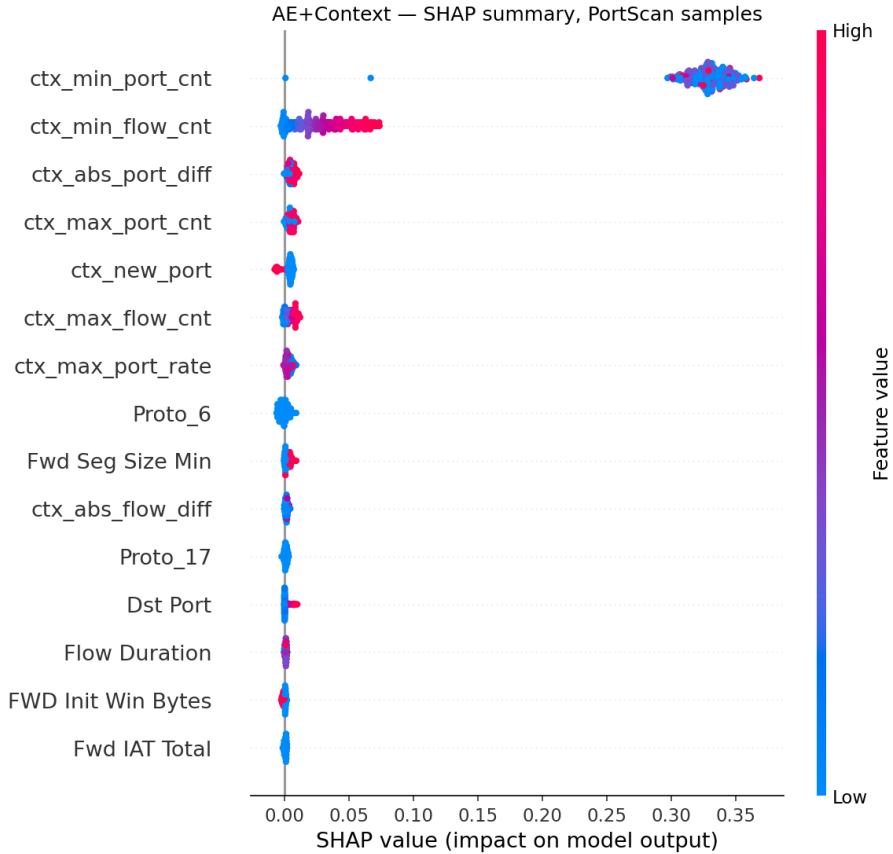


Figure 5.7: AE+Context – SHAP summary plot, PortScan samples. `ctx_min_port_cnt` dominates with a tight, uniformly positive-SHAP cluster well separated from every other feature; the remaining top rows are almost entirely `ctx_*` host-context features, with ordinary flow features only appearing near the bottom of the ranking.

	<code>ctx_*</code> features in top 15	Leading feature (mean $ \text{SHAP} $)
PortScan	8/15	<code>ctx_min_port_cnt</code> (0.324)
DDoS	7/15	<code>ctx_min_port_cnt</code> (0.234)

For both classes, `ctx_min_port_cnt` and `ctx_min_flow_cnt` dominate by roughly an order of magnitude over every ordinary flow feature: on PortScan, `ctx_min_port_cnt`’s attribution (0.324) is nearly $70\times$ the next-highest ordinary feature, `Fwd Seg Size Min` (0.0023); on DDoS it is over $6\times$ the leading ordinary feature, `Bwd Packet Length Std`

(0.035). Six of the nine `ctx_*` features appear in both classes’ top-15 lists, with only `ctx_min_port_rate` absent from both. This confirms the mirror premise: AE+Context’s advantage on these classes is driven almost entirely by host-context evidence unavailable to Temporal IForest, not by the ordinary flow features the two models share.

Taken together with §5.2.3’s result for Temporal IForest, the two halves of the disjoint-feature-regime premise both hold: Temporal IForest’s strong classes are explained by evidence spread across the 16-flow window (packet-length statistics for volumetric floods, inter-arrival-time statistics for slow-drip attacks), and AE+Context’s strong classes are explained by host-context features that exist only in its own feature set. The proposed ensemble’s complementary coverage is therefore backed by genuinely non-overlapping evidence between its two members, not by two rankings that merely happen to differ on which classes they cover.

5.2.5 Building the Combined Score

Confirming the disjoint-evidence premise does not by itself establish that combining the two scores works in practice. Realizing the ensemble sketched in Chapter 4 requires pairing, for every flow f in the test set, AE+Context’s own per-flow score $s_{AE}(f)$ with Temporal IForest’s score for the 16-flow window *ending* at f – the same causal, non-leaking choice validated in the previous subsection via last-flow labeling. Since the two preprocessing pipelines reorder flows differently before saving their respective test arrays (AE+Context sorts globally by timestamp; Temporal IForest sorts per source-IP host), the two score arrays are not aligned index-for-index and must be re-aligned by flow identity, recovering each flow’s original row position in both pipelines’ shared, pre-sort input.

Once aligned, the two raw scores cannot be combined directly: AE+Context’s score is a mean-squared reconstruction error with no fixed scale, and Temporal IForest’s score is a negated mean path length, and the two live on entirely different numeric ranges. Some normalization is required before either a weighted average or a max can be meaningful. Each model’s normalization statistics are computed on that same model’s own *benign training* scores only, never on test data, to avoid leaking test-set anomaly information into the fusion.

5.2.6 Two Fusion Rules, Two Normalizations

Two combination rules were tested. **Weighted soft voting** takes a convex combination of the two normalized scores, $s_{\text{soft}}(f) = w \cdot \hat{s}_{AE}(f) + (1 - w) \cdot \hat{s}_{IF}(f)$, swept over $w \in \{0.3, \dots, 0.7\}$. **Max-rule fusion** instead takes $s_{\text{max}}(f) = \max(\hat{s}_{AE}(f), \hat{s}_{IF}(f))$ – an OR-like combiner intended to match the disjoint-evidence story more directly: either model’s confidence alone should be enough to flag a flow.

The first normalization attempted, standard z -scoring against each model’s own benign training mean and standard deviation, turned out to be a dead end rather than a genuine test of either fusion rule. AE+Context’s reconstruction-error z -scores are extremely heavy-tailed (maximum $z \approx 2366$ on the test set, with the 99.9th percentile of *benign* flows alone already at $z \approx 227$), while Temporal IForest’s z -scores stay tightly bounded (maximum $z \approx 21$). Under z -score normalization, a handful of extreme AE+Context outliers dominate any linear combination or max regardless of the weight assigned to Temporal IForest: max-rule and every soft-vote weight tested reproduced AE+Context’s own per-class ranking almost exactly (e.g. GoldenEye AP stayed at 0.215–0.216 across all five weights, against AE+Context alone’s 0.215), rather than showing any influence from Temporal IForest’s much stronger 0.627 standalone score on that class. This is a normalization artifact, not evidence that fusion does not help – it does not actually test the ensemble hypothesis.

Replacing z -score with rank normalization – mapping each test score to its percentile within that model’s own benign training-score distribution, via the empirical CDF – removes this problem by construction: every normalized score is bounded to $[0, 1]$ regardless of how extreme the raw value is, so neither model’s tail can dominate the other’s by sheer magnitude. All results below use rank normalization.

5.2.7 Per-Class Coverage vs. Aggregate AP

	DDoS	PortScan	GoldenEye	Slowhttpstest	slowloris
AE+Context alone	0.882	0.924	0.292	0.013	0.025
Temporal IForest alone	0.956	0.123	0.611	0.578	0.560
Max-rule (rank)	0.882	0.923	0.292	0.060	0.083
Soft-vote $w = 0.4$ (rank)	0.977	0.228	0.801	0.205	0.319
Soft-vote $w = 0.7$ (rank)	0.977	0.358	0.821	0.140	0.245

Aggregate AP over the full row-aligned test set actually *favors* the variants that change the least: max-rule (aggregate AP 0.943) and AE+Context alone (0.941) both edge out soft-vote $w = 0.4$ (0.841), simply because DDoS and PortScan together account for over 250,000 of the roughly 434,000 positive flows in this evaluation, while GoldenEye, Slowhttpstest, and slowloris combined account for barely 13,000. An aggregate metric dominated by two classes’ raw volume is exactly the wrong lens for judging whether an ensemble motivated by *per-class coverage* actually delivers it, so §4.3.2’s per-class breakdown, not the aggregate table, is the right basis for evaluating this ensemble – consistent with how every other per-attack-type claim in this thesis is judged.

Read per-class, max-rule falls short of its own premise: because it is an OR over two scores, it inherits the *union* of both models’ false-positive tendencies rather than only their

true positives, so on GoldenEye/Slowhttptest/slowloris it barely moves off AE+Context’s own weak scores ($0.292 \rightarrow 0.292$, $0.013 \rightarrow 0.060$, $0.025 \rightarrow 0.083$) despite Temporal IForest’s much stronger standalone numbers on exactly these classes. Soft voting shows the opposite failure mode on PortScan: because it averages rather than maxes, and Temporal IForest is actively poor on PortScan (0.123 , well below AE+Context’s 0.924) rather than merely uninformative, blending pulls PortScan’s ranking down substantially ($0.924 \rightarrow 0.228$ at $w = 0.4$) even while lifting GoldenEye from 0.292 to 0.801 .

Neither rule is a free win, but soft voting is the more defensible choice for a coverage-motivated ensemble: its *weakest* covered class (0.205 , Slowhttptest, at $w = 0.4$) clears both standalone models’ own weakest classes – AE+Context’s 0.013 (Slowhttptest) and Temporal IForest’s 0.123 (PortScan) – something neither standalone model nor max-rule fusion achieves. This is a genuine, if partial, realization of the ensemble idea from Chapter 4: rank-normalized soft voting measurably widens the floor of attack-class coverage, at the explicit cost of PortScan detection quality, rather than the free lunch the original aggregate-level framing implied. A deployment that must retain AE+Context’s near-complete PortScan coverage would instead need a class-aware combination rule (e.g. routing PortScan-like flows to AE+Context alone) rather than a single global weight – left as future work.

Capitolul 6

Concluzii

Anexa 1

Alegeri legate de Hiperparametri și Antrenare

Două decizii principale se aplică uniform pentru toate modelele și seturile de date:

- **Optimizator.** Toate rețelele neuronale sunt antrenate cu Adam, la o rată de învățare fixă de 10^{-3} , uniform pentru fiecare model și set de date.
- **Fără căutare de hiperparametri.** Dacă un model a avut performanțe neașteptat de slabe, rata sa de învățare a fost verificată suplimentar cu $\{10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$ înainte de a trage vreo concluzie din acel rezultat.

- **Partiție de validare doar pentru monitorizare.** Fiecare rețea neuronală reține aleator 10% din datele de antrenare exclusiv pentru a trasa o curbă de pierdere (*loss*) de antrenare/validare per rulare, ca verificare că antrenarea decurge corect. Această partiție nu este niciodată folosită pentru oprire timpurie (*early stopping*) sau selecția modelului: antrenarea rulează întotdeauna pentru numărul fix de epoci de mai jos, iar rezultatele raportate pe setul de testare folosesc ponderile modelului din epoca finală.

1.1 Isolation Forest

	CIC flat	CIC context	Yahoo	Credit card
N_ESTIMATORS	150	150	150	150
MAX_SAMPLES	300,000	300,000	$\min(n, 50,000)$	200,000
Dimensiunea setului de antrenare	~371k	~371k	câteva mii	~227k
% din antrenare folosit	~81%	~81%	~100%	~88%
RANDOM_SEED	123	123	123	123

MAX_SAMPLES este singura diferență semnificativă și este setat cât mai ridicat posibil, practic, pentru fiecare set de date (acuratețea crește odată cu mai multe eșantioane per arbore, cu randamente descrescând; problema de „swamping” al arborilor nu apare aici, întrucât datele de antrenare sunt întotdeauna exclusiv benigne). Plafonul de 50.000 pentru Yahoo nu este atins niciodată în practică și este păstrat doar ca limită de siguranță.

1.2 One-Class SVM

	CIC flat	CIC context	Yahoo	Credit card
NU	0.01	0.01	0.01	0.01
KERNEL	rbf	rbf	rbf	rbf
GAMMA	scale	scale	scale	scale
SUBSAMPLE	50,000	50,000	niciunul (complet)	niciunul (complet)
% din antrenare folosit	~13.5%	~13.5%	~100%	~100%

Toți parametrii kernel/nu sunt uniformi pe seturile de date. Subeșantionarea este aplicată doar acolo unde setul de antrenare este suficient de mare încât un OC-SVM pe toate datele ar deveni mult prea lent; credit card (~227k rânduri) a fost antrenat pe tot setul, fără subeșantionare.

1.3 Autoencoder Feedforward (MLP)

	CIC flat	CIC context	Yahoo	Credit card
Dimensiune intrare	77	86	64 (fereastră aplatizată)	~30 (PCA)
Dimensiuni ascunse	[64, 32, 16]	[64, 32, 16]	[32, 16]	[16, 8]
Raport bottleneck	0.21	0.19	0.25	~0.27
Activare finală	Sigmoid	Sigmoid	Linear	Linear

Lăţimile straturilor sunt scalate în funcţie de dimensiunea intrării, astfel încât raportul de compresie al bottleneck-ului să rămână aproximativ constant (~ 0.20 – 0.27) pe toate seturile de date. Activarea finală diferă întrucât CIC foloseşte **MinMaxScaler** (mărginit la $[0, 1]$, potrivit pentru o ieşire sigmoid), în timp ce Yahoo şi credit card folosesc scalere nemărginite (ieşire liniară).

1.4 LSTM Predictiv

	CIC	Yahoo	Credit card
INPUT_SIZE	77	1	1
HIDDEN_DIM	128	64	64
NUM_LAYERS	1	1	1
EPOCHS	20	20	50
BATCH_SIZE	256	256	512

HIDDEN_DIM este scalat în funcţie de dimensiunea intrării pentru CIC (o extindere de $\sim 1.66\times$ faţă de cele 77 de caracteristici de intrare); ambele seturi de date univariate folosesc 64, o capacitate suficientă pentru un semnal scalar. Credit card foloseşte mai multe epoci (50) întrucât curba sa de pierdere continuă să scadă şi după epoca 30 – probabil o consecinţă directă a tratării celor 29 de caracteristici PCA netemporale ca pseudo-secvenţă, ceea ce face optimizarea mai zgomotoasă decât pe date cu adevărat temporale. CIC şi Yahoo converg până la epoca ~ 5 , deci 20 de epoci este deja o alegere conservatoare pentru acestea.

1.5 Autoencoder LSTM

	CIC	Yahoo	Credit card
INPUT_SIZE	77	1	1
HIDDEN_DIM	128	64	64
NUM_LAYERS	1	1	1
EPOCHS	50	30	50
BATCH_SIZE	256	256	512

Atât CIC, cât și credit card au necesitat 50 de epoci pentru a atinge un platou; Yahoo converge până la epoca ~ 5 , deci 30 este deja generos.

1.6 Bottleneck Transformer Autoencoder

	CIC	Yahoo	Credit card
D_MODEL	64	64	64
NHEAD	4	4	4
NUM_LAYERS	2	2	2
DIM_FF	256	256	256
LATENT_DIM	32	32	32
EPOCHS	50	20	20
BATCH_SIZE	256	256	512

LATENT_DIM=32 oferă o compresie uniformă de 50% față de D_MODEL=64, pe toate cele trei seturi de date.

1.7 Causal Transformer

	CIC	Yahoo
D_MODEL	64	64
NHEAD	4	4
NUM_LAYERS	2	2
DIM_FF	256	256
DROPOUT	0.1	0.1
EPOCHS	30	30
BATCH_SIZE	256	256

Anexa 2

Protocolul de Evaluare: Metrici și Rezultate Complete

2.1 Metrici Raportate

Fiecare rulare produce un raport text care conține toate metricile din Tabelul 2.1. Capitolul de Rezultate raportează doar un subset dintre acestea (de obicei ROC-AUC, Precizia Medie, și, unde este relevant, Acuratețea Echilibrată și F1), ales pentru a nu aglomera tabelele de comparație; setul complet, per model și per set de date, este prezentat în §2.2–§2.4.

Metrică	Note
Precizie / Recall / F1 (per clasă)	Prin <code>sklearn.classification_report</code> , raportate atât pentru clasa normală, cât și pentru clasa anomalie, plus acuratețea și mediile macro/ponderate.
ROC-AUC	Raportat pentru completitudine / comparabilitate cu literatura anterioară; nu este metrica principală de clasificare în condiții de dezechilibru de clase.
Precizie Medie (PR-AUC)	Independentă de prag; alegerea standard în condiții de dezechilibru sever de clase, întrucât ROC-AUC poate părea înșelător de bun atunci când exemplele negative depășesc masiv numărul celor pozitive.
Acuratețe Echilibrată	$\frac{1}{2}$ (sensibilitate + specificitate) la pragul de operare selectat.
F1 (clasa pozitivă/atac)	La pragul de operare selectat.
Numărători din matricea de confuzie	TP/TN/FP/FN brute, la pragul selectat.

Tabela 2.1: Metrici raportate de fiecare rulare de evaluare.

Notă privind consistența numerelor. Tabelele 2.2–2.4 de mai jos completează Capitolul 4 cu detaliul complet (precizie/recall pe clasa pozitivă, matrice de confuzie) pentru fiecare model. Coloanele ROC-AUC / Precizie Medie / Acuratețe Echilibrată / F1 repro-

duc exact valorile deja publicate în Capitolul 4, pentru consistență. Pentru un număr de modele – în special pe CIC-IDS2017, unde reevaluări ulterioare au schimbat scorurile – nu toate metricile au mai fost calculate. În aceste cazuri, rândul din tabel afișează doar metricile agregate deja publicate.

2.2 Rezultate Complete – Credit Card Fraud Detection

Model	Precizie (fraudă)	Recall (fraudă)	F1 (fraudă)	ROC-AUC	Precizie Medie	Acuratețe Echilibrată	TP	TN	FP	FN
MLP Autoencoder	0.6205	0.8008	0.6992	0.9649	0.7105	0.8983	394	56622	241	98
Isolation Forest	0.5000	0.7724	0.6070	0.9587	0.6725	0.8828	380	56483	380	112
LSTM Autoencoder	0.6904	0.7886	0.7362	0.9476	0.6223	0.8928	388	56689	174	104
OC-SVM	0.5501	0.7805	0.6454	0.9354	0.5959	0.8875	384	56549	314	108
Transformer Bottleneck AE	0.5073	0.7764	0.6137	0.9583	0.5947	0.8849	382	56492	371	110
Predictive LSTM	0.3992	0.7642	0.5244	0.9581	0.5190	0.8771	376	56297	566	116

Tabela 2.2: Credit Card Fraud Detection – toate metricile raportate, pentru toate cele șase modele evaluate.

2.3 Rezultate Complete – Yahoo Webscope S5

Model	Precizie (poz.)	Recall (poz.)	F1 (poz.)	ROC-AUC	Precizie Medie	Acuratețe Echilibrată	TP	TN	FP	FN
A1 – anomalii punctuale										
MLP Autoencoder	–	–	–	0.792	0.611	–	–	–	–	–
LSTM Autoencoder	0.2944	0.7904	0.4290	0.794	0.600	0.6916	852	2974	2042	226
OC-SVM	0.5183	0.5909	0.5522	0.776	0.595	0.7364	637	4424	592	441
LSTM Predictiv	0.2988	0.8016	0.4354	0.789	0.588	0.6988	3442	11912	8076	852
Isolation Forest	0.2439	0.8646	0.3805	0.765	0.511	0.6443	932	2127	2889	146
A3 – anomalii contextuale/sezoniere										
LSTM Predictiv	–	–	–	0.868	0.812	–	–	–	–	–
Transformer Bottleneck AE	–	–	–	0.762	0.663	–	–	–	–	–
Causal Transformer	–	–	–	0.738	0.623	–	–	–	–	–
MLP Autoencoder	0.4247	0.9540	0.5878	0.765	0.507	0.6622	13785	10981	18670	664
LSTM Autoencoder	0.3278	1.0000	0.4937	0.566	0.392	0.5003	14449	20	29631	0
OC-SVM	0.3285	0.9996	0.4945	0.584	0.376	0.5020	14443	128	29523	6
Isolation Forest	0.3301	0.9960	0.4959	0.511	0.331	0.5055	14391	447	29204	58
A4 – schimbări de nivel										
LSTM Predictiv	0.3975	0.7951	0.5300	0.745	0.505	0.6864	9098	18868	13789	2345
Causal Transformer	0.3092	0.9098	0.4616	0.695	0.436	0.5988	10411	9401	23256	1032
MLP Autoencoder	–	–	–	0.719	0.402	–	–	–	–	–
Transformer Bottleneck AE	0.2625	0.9959	0.4155	0.655	0.404	0.5077	11396	636	32021	47
Isolation Forest	0.2595	1.0000	0.4121	0.498	0.268	0.5001	11443	6	32651	0
OC-SVM	0.2603	1.0000	0.4131	0.519	0.265	0.5021	11443	139	32518	0
LSTM Autoencoder	0.2644	0.9979	0.4181	0.502	0.252	0.5127	11419	895	31762	24

Tabela 2.3: Yahoo Webscope S5 – toate metricile raportate, pe benchmark (A1, A3, A4), ordonate ca în tabelele din Capitolul 4.

2.4 Rezultate Complete – CIC-IDS2017

Compoziția pe clase de atac a setului de test temporal (analoagă Tabelului 4.5 din Capitolul 4 pentru setul flat/context) este detaliată separat, în Capitolul 3.

Model	Set de caracteristici	Precizie (atac)	Recall (atac)	F1 (atac)	ROC-AUC	Precizie Medie	Acuratețe Echilibrată	TP	TN	FP	FN
MLP Autoencoder	Context	–	–	–	0.9860	0.9487	0.9548	–	–	–	–
OC-SVM	Context	0.8965	0.9509	0.9229	0.9745	0.9396	0.9569	412650	1238310	47634	21327
Isolation Forest	Context	–	–	–	0.9785	0.9328	0.9270	–	–	–	–
LSTM Autoencoder	Temporal	0.4752	0.9126	0.6249	0.8976	0.7955	0.7846	398793	842158	440453	38212
MLP Autoencoder	Flat	–	–	–	0.9189	0.7921	0.8022	–	–	–	–
Isolation Forest	Flat	–	–	–	0.9049	0.7612	0.8228	–	–	–	–
OC-SVM	Temporal	0.4214	0.9770	0.5888	0.8629	0.7816	0.7600	426967	696384	586227	10038
MLP Autoencoder	Temporal	–	–	–	0.8831	0.7509	0.7734	–	–	–	–
Isolation Forest	Temporal	0.3577	0.9487	0.5195	0.8438	0.7604	0.6841	414606	538115	744496	22399
OC-SVM	Flat	0.3045	0.9778	0.4644	0.7336	0.6845	0.6121	424338	316815	969129	9639
Bottleneck Transformer AE	Temporal	–	–	–	0.8077	0.6539	0.7059	–	–	–	–
Causal Transformer	Temporal	–	–	–	0.7099	0.5449	0.5520	–	–	–	–
Predictive LSTM	Temporal	–	–	–	0.6126	0.4472	0.5475	–	–	–	–

Tabela 2.4: CIC-IDS2017 – toate metricile raportate, pentru toate cele treisprezece combinații model/set-de-caracteristici, ordonate ca în Tabelul din §4.

Anexa 3

Compoziția pe Clase de Atac a Setului de Test Temporal CIC-IDS2017

Tabelul 3.1 detaliază pe clase ferestrele de atac agregate din setul de test al pipeline-ului temporal, analog Tabelului 4.5 din Capitolul 4 pentru setul flat/context. DoS Hulk este inclus aici din același motiv discutat acolo: fluxurile sale rămân parte a clasei pozitive pentru orice metrică agregată, deși este exclus din detalierea pe clasă din secțiunea 4.3.2.

Clasă de atac	n (ferestre)	% din grupul de atacuri
PortScan	159,160	36.42%
DoS Hulk	158,469	36.27%
DDoS	95,131	21.77%
DoS GoldenEye	7,575	1.73%
DoS slowloris	4,001	0.92%
FTP-Patator	3,966	0.91%
Bot	3,320	0.76%
SSH-Patator	2,980	0.68%
DoS Slowhttptest	1,742	0.40%
Infiltration	445	0.10%
Web Attack – Brute Force	151	0.03%
Web Attack – XSS	26	0.01%
Web Attack – SQL Injection	20	<0.01%
Heartbleed	19	<0.01%
BENIGN	1,282,611	–

Tabela 3.1: CIC-IDS2017 – compoziția pe clase de atac a ferestrelor de atac agregate din setul de test temporal (1,719,616 ferestre în total, rată de atac de 25.41%).